



WEB FRONTEND DEVELOPMENT TECHNOLOGY

Revision Notes

HTML5 | CSS3 | Tables | Forms
JavaScript | jQuery | Bootstrap | Vue 3



1. HTML5 Document Structure

HTML is the language used to define the structure and content of a webpage. A valid HTML5 page begins with a document type declaration, then the root element, then the head and body sections.

Example: Basic HTML5 page structure

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Document Title</title>
  <link rel="stylesheet" href="styles.css">
  <script src="app.js"></script>
</head>
<body>
  <h1>Welcome</h1>
  <p>Visible content goes here.</p>
</body>
</html>
```

Actual browser output

Browser tab: Document Title

Welcome

Visible content goes here.

Part	Role
<!DOCTYPE html>	Declares HTML5 and tells the browser to use standards mode. It should appear first.
<html>	Root element that wraps the complete HTML document.
<head>	Contains metadata, title, links to CSS, scripts, charset, and other information not normally shown as page content.
<title>	Sets the text visible on the browser tab or page title area.
<body>	Contains visible content such as text, headings, images, links, tables, forms, and components.

Key point

The <head> section stores metadata and resource links. The <body> section stores what users actually see in the browser window.

2. Common HTML Text, Link, Image, List, and Media Tags

Tag or feature	Meaning / use
<h1> to <h6>	Headings from largest/most important (<h1>) to smallest (<h6>).
<p>	Paragraph text.
 	Line break. It does not insert an image.
, <i>, <u>, <p>	Bold, italic, underline, and paragraph text elements.
<div>	Block-level container by default; starts on a new line and usually takes full available width.
	Hyperlink. The href attribute specifies the destination page or website.
	Displays an image using the src attribute. Common web formats include GIF, JPEG/JPG, and PNG.
, ,	Ordered list, unordered list, and list item.
<meter>	Shows a scalar measurement within a known range, such as disk usage or score.

Tag or feature	Meaning / use
<details> and <summary>	Creates expandable/collapsible content. <summary> is the clickable visible heading.
<iframe>	Embeds another webpage inside the current page.

Ordered and unordered lists are used for different purposes:

Example: Ordered list and unordered list

```
<ol>
  <li>Wake up</li>
  <li>Brush teeth</li>
  <li>Eat breakfast</li>
</ol>

<ul>
  <li>Apples</li>
  <li>Bananas</li>
  <li>Oranges</li>
</ul>
```

Actual browser output

Browser tab: List output

1. Wake up
2. Brush teeth
3. Eat breakfast

- Apples
- Bananas
- Oranges

Use an ordered list when sequence matters. Use an unordered list when the items are a group but the order is not important.

Media attribute	Use
controls	Displays play, pause, volume, and other browser controls.
autoplay	Starts playing automatically when supported by the browser.
loop	Repeats the media after it ends.
muted	Starts the media without sound.

3. Semantic HTML5 Tags

Semantic tags clearly describe the meaning of content. They improve readability, accessibility, and search-engine understanding. Unlike old presentational tags such as , semantic tags explain the page structure.

Semantic tag	Purpose
<header>	Introductory area of a page or section, often containing logo, title, or navigation.
<nav>	Navigation links such as menu items or table of contents.
<article>	Independent, self-contained content such as a blog post or news item.
<aside>	Side content related to the main content, such as notes, ads, or sidebars.
<section>	Thematic grouping of related content.
<footer>	Footer information such as copyright, contact, or related links.

Important distinction

Semantic structure tags include `<nav>`, `<article>`, and `<aside>`. The old `<font>` tag is not a semantic HTML5 structure tag.

4. CSS Purpose, Types, Priority, and Selectors

CSS stands for Cascading Style Sheets. It controls the appearance, layout, and presentation of HTML elements. CSS separates content from design, making pages easier to maintain and style consistently.

CSS type	Where it is written	Example	Priority idea
Inline CSS	Inside an element using the style attribute	<code><p style="color:red;">Text</p></code>	Highest priority for the same element and property.
Internal CSS	Inside a <code><style></code> tag in the <code><head></code> section	<code><style> p { color: blue; } </style></code>	Applies to one page; usually higher than external CSS if selectors are equally specific and later in cascade.
External CSS	In a separate .css file linked using <code><link></code>	<code><link rel="stylesheet" href="styles.css"></code>	Best for multi-page websites and reusable design.

Priority shortcut

A useful cascade shortcut is: inline CSS > internal CSS > external CSS > browser default. Specificity, order, and !important can also affect real CSS behaviour.

Selectors tell CSS which elements to style.

Selector	Meaning	Example
Element selector	Selects all elements with a tag name	<code>p { color: red; }</code>
Class selector	Selects elements with a class. It starts with a dot .	<code>.note { color: blue; }</code>
ID selector	Selects one unique element. It starts with #	<code>#main { width: 80%; }</code>
Descendant selector	Selects nested elements	<code>div p</code>
Child selector	Selects direct children only	<code>div > p</code>
Adjacent sibling	Selects the next sibling	<code>div + p</code>
General sibling	Selects following siblings	<code>div ~ p</code>

5. CSS Properties for Text, Fonts, Lists, Background, Display, and Position

Category	Properties / values to remember
Text formatting	text-align, text-decoration, text-transform, letter-spacing
Font styling	font-family, font-size, font-style, font-weight
Text color	color changes the foreground text colour.
Background styling	background-color and background-image set background appearance.
List markers	list-style-type values include disc, circle, square, and decimal.
Table appearance	border, border-collapse, padding, text-align

Display and visibility are frequently confused:

CSS rule	What happens
display: none	The element is hidden and removed from the layout. Its space is not reserved.
visibility: hidden	The element is hidden but its original space is still reserved.
display: block	The element behaves as a block-level element.

The position property controls how an element is placed.

position value	Meaning
static	Default normal page flow.
relative	Positioned relative to its normal position.
absolute	Positioned relative to the nearest positioned ancestor.
fixed	Positioned relative to the viewport and stays in place when the page scrolls.

6. HTML Tables

Tables display data in rows and columns. A table can include a caption, header cells, data cells, and grouped sections.

Table item	Role
<table>	Main table container.
<caption>	Title or description for the table.
<tr>	Table row.
<th>	Header cell, usually bold and centered by default.
<td>	Normal table data cell.
<thead>, <tbody>, <tfoot>	Organize table header, body, and footer sections.
colspan	Merges a cell horizontally across multiple columns.
rowspan	Merges a cell vertically across multiple rows.

Example: Table with caption, colspan, and rowspan

```
<table border="1">
  <caption>Student Marks</caption>
  <tr>
    <th>Name</th>
    <th colspan="2">Scores</th>
  </tr>
  <tr>
    <td rowspan="2">John</td>
    <td>Math: 90</td>
    <td>Science: 85</td>
  </tr>
  <tr>
    <td colspan="2">Total: 175</td>
  </tr>
</table>
```

Actual browser output**Browser tab:** Table output**Student Marks**

Name	Scores	
John	Math: 90	Science: 85
	Total: 175	

Concept link

colspan goes across columns, so it works horizontally. rowspan goes across rows, so it works vertically.

7. HTML Forms and Validation

Forms collect user input and submit it to a server or process it with JavaScript. The `<form>` tag usually works with attributes such as `action`, `method`, `target`, and `enctype`.

Form concept	Explanation
<code><form></code>	Main container used to collect and submit user input.
GET method	Appends form data to the URL. Useful for simple, non-sensitive requests.
POST method	Sends form data in the request body. Better for larger or sensitive data.
required	Makes a field mandatory before submission.
placeholder	Displays a hint inside the field.
pattern	Defines a regular-expression pattern for validating input.
datalist	Provides predefined suggestions for an input field.
<code><optgroup></code>	Groups related options inside a drop-down list.
Control	Use
<code><input type="text"></code>	Single-line text such as name or address.
<code><input type="password"></code>	Hides typed characters for password entry.
<code><input type="radio"></code>	Lets the user select only one option from a group sharing the same name.
<code><input type="checkbox"></code>	Lets the user select one or more options.
<code><input type="email"></code>	Collects email address and provides built-in validation.
<code><input type="url"></code>	Collects URL and provides built-in validation.
<code><input type="number"></code>	Collects numeric input and can use min/max.
<code><input type="date"></code>	Collects a date value using the browser date control when supported.
<code><input type="submit"></code>	Creates a submit button for sending form data.
<code><input type="tel"></code>	Collects telephone numbers; validation depends on extra rules.
<code><select></code>	Creates a drop-down list; <code><optgroup></code> can group related options.

Example: Form validation using HTML and JavaScript

```
<form id="myForm" method="post">
  <input type="text" id="name" required placeholder="Name">
  <input type="email" id="email" required placeholder="Email">
  <input type="number" id="age" min="18" required>
  <button type="submit">Submit</button>
```

```

</form>

<script>
document.getElementById("myForm").onsubmit = function(e) {
  let age = document.getElementById("age").value;
  if (age < 18) {
    alert("Age must be 18 or above.");
    e.preventDefault();
  }
};
</script>

```

Actual browser output

Browser tab: Form output

Name	Name
Email	Email
Age	17
Submit	

Browser alert: Age must be 18 or above.

Validation idea

HTML attributes such as required, type="email", type="number", and pattern provide browser-level validation. JavaScript is used for custom rules and dynamic checks before submission.

8. JavaScript and jQuery Essentials

JavaScript adds behaviour to web pages. It can read user input, validate forms, change HTML/CSS dynamically, and respond to events. Basic building blocks include variables, functions, loops, and conditions.

Concept	Explanation
document.getElementById("id")	Common JavaScript method for selecting an element by id.
Variables	Store data values.
Functions	Reusable blocks of code.
Loops	Repeat code.
Conditions	Run code depending on true/false logic.

jQuery is a JavaScript library that simplifies DOM selection, events, and effects.

jQuery syntax	Meaning
\$(document).ready(function(){ ... })	Runs code only after the DOM tree has been fully constructed.
\$("#myButton").click(function(){ ... })	Attaches a click handler to the selected element.
\$("#myButton").click()	Triggers the click event programmatically when no function is passed.
.slideToggle()	Slides an element up or down, toggling between hidden and visible.

Important note

.slideToggle() is not only for sliding up. It toggles: if the element is visible, it slides up; if hidden, it slides down.

9. Bootstrap Essentials

Bootstrap is a front-end framework that provides ready-made CSS classes and components for responsive design. It uses a 12-column grid system.

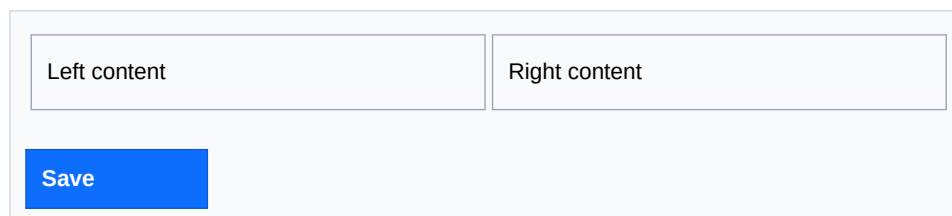
Bootstrap item	Use
.container	Responsive fixed-width container.
.container-fluid	Full-width container.
.row	Wraps grid columns; columns are usually placed inside a .row.
.col-*	Grid columns in a 12-column responsive layout.
.btn	Basic button class.
.btn-primary, .btn-success, .btn-danger	Button colour/context classes.
.btn-sm	Creates a smaller button size.
.btn-block	Older Bootstrap class for a full-width block button.
.navbar	Navigation bar for grouped navigation links.
.badge	Small label/count indicator.
.progress	Progress bar container.
Tooltip	Small helper text displayed on hover/focus.
.form-control	Styles input fields, select lists, and textareas.
.form-group	Common Bootstrap 4 grouping class for form controls.
.form-check	Styles checkbox and radio controls.
.form-inline	Common Bootstrap 4 class for inline forms.
.form-label, .input-group	Useful Bootstrap label and input-group classes.

Example: Bootstrap grid and button

```
<div class="container-fluid">
  <div class="row">
    <div class="col-md-6">Left content</div>
    <div class="col-md-6">Right content</div>
  </div>
  <button class="btn btn-primary">Save</button>
</div>
```

Actual browser output

Browser tab: Bootstrap output



10. Vue 3 and Vue CLI

Vue.js is a progressive JavaScript framework for building user interfaces. Vue 3 commonly uses components, reactivity, directives, and a virtual DOM to create efficient and maintainable web applications.

Vue feature	Meaning
Reactivity system	Vue tracks data changes and updates the DOM automatically.

Vue feature	Meaning
Component-based architecture	UI is built using reusable components with their own template, logic, and style.
Declarative rendering	Templates describe what the UI should look like based on data.
Directives	Special attributes such as v-if, v-for, v-bind, v-on, and v-model.
Two-way data binding	v-model keeps form input and component data synchronized.
Virtual DOM	Vue calculates efficient UI updates before changing the real DOM.

Directive / option	Use
data()	Returns the reactive state object for each component instance. Each instance should have separate data.
{{ }} interpolation	Displays data in the template.
v-bind	Binds data to HTML attributes. Shorthand is :.
v-on	Handles events. Shorthand is @.
v-model	Creates two-way binding with form controls.
v-for	Renders a list by iterating over an array.
v-if	Conditionally renders an element.
v-else / v-else-if	Alternative conditional branches.
v-show	Conditionally displays an element using CSS display.
v-once	Renders an element once and then skips future updates.

v-once correction

v-once does not force re-rendering. It does the opposite: it renders the element once and then keeps it static, even if related data changes.

Component registration can be global or local:

Registration type	Meaning
Global registration	Done with app.component("ComponentName", Component) in main.js. The component is available throughout the app.
Local registration	Component is available only inside the component where it is imported and registered. It is recommended for most components to reduce naming conflicts and unnecessary global scope.

Common Vue CLI commands and project files:

Item	Purpose
vue create project-name	Creates a Vue CLI project.
npm run serve	Runs the development server.
App.vue	Root component in a Vue CLI project.
main.js	Entry file where the Vue app is created and mounted.
src folder	Main source code folder for components and application logic.
public folder	Static assets and public index file.
package.json	Project scripts and dependency information.
node_modules	Installed project dependencies.

Example: Vue component using v-model, interpolation, and v-for

```
<template>
  <input v-model="name" placeholder="Enter name">
```

```
<p>Hello {{ name }}</p>
<ul>
  <li v-for="item in items" :key="item.id">{{ item.text }}</li>
</ul>
</template>

<script>
export default {
  data() {
    return {
      name: '',
      items: [
        { id: 1, text: 'HTML' },
        { id: 2, text: 'CSS' },
        { id: 3, text: 'Vue' }
      ]
    }
  }
}
</script>
```

Actual browser output

Browser tab: Vue output

Hello Siddaram

- HTML
- CSS
- Vue

If the input value is changed, the greeting updates immediately because of v-model.

Getting Started with HTML

Chapter 1 · Web Technology – structural & semantic tags, multimedia, metadata

1. Basic structure of an HTML page

structural tags

Every HTML document follows a fixed skeleton.

Minimal HTML5 template

```
<!DOCTYPE HTML>
<HTML>
  <HEAD>
    <TITLE>Home Page of Book Your Hotel</TITLE>
  </HEAD>
  <BODY>
    Book your favorite hotel.
  </BODY>
</HTML>
```

browser output (chrome / edge)

Book your favorite hotel.

TAG	TYPE	DESCRIPTION
<!DOCTYPE HTML>	empty	Instructs browser about HTML5 version; must be first.
<HTML> ... </HTML>	container	Root element – wraps entire page.
<HEAD> ... </HEAD>	container	Header section: meta, title, styles, scripts.
<BODY> ... </BODY>	container	Contains visible content (text, images, links).

container vs empty: Container (paired) tag needs closing `</...>`. Empty tag has no content & no closing (e.g. `<!DOCTYPE HTML>`, `<BR>`).

2. Exploring the `<HEAD>` section

The `<HEAD>` can contain `<TITLE>`, `<META>`, `<STYLE>`, `<LINK>`, `<SCRIPT>`, `<BASE>`, `<NOSCRIPT>`.

`<TITLE>` - browser tab & search results

```
<HEAD>
  <TITLE>Book Your Hotel</TITLE>
</HEAD>
```

title bar shows: "Book Your Hotel" (only one per doc)

`<META>` - metadata for search engines

```
<HEAD>
  <META name="description" content="Best hotel booking in US">
  <META name="keywords" content="hotel, online booking, rooms">
  <META name="author" content="Harry Anem">
  <META http-equiv="refresh" content="30">
</HEAD>
```

ATTRIBUTE	EXAMPLE VALUE	PURPOSE
<code>name="description"</code>	<code>"online hotel booking"</code>	short summary for search engines
<code>name="keywords"</code>	<code>"hotel, booking"</code>	relevant search keywords
<code>name="author"</code>	<code>"Harry Anem"</code>	document author
<code>http-equiv="refresh"</code>	<code>content="30"</code>	auto-refresh every 30 seconds

`<LINK>` - external relationships

```
<LINK href="StyleHome.css" rel="stylesheet">
```

Attributes: `rel`, `href`, `hreflang`, `media` (screen, handheld, tv ...).

`<SCRIPT>` - client-side scripting

```
<SCRIPT type="text/javascript">
  var name = prompt("Please Enter Your Name","John");
  alert(name);
</SCRIPT>
```

Attributes: `async`, `defer`, `src`, `type`.

`<BASE>` - base URL for relative links

```
<BASE href="d:/assets/img/">
<!-- then <IMG src="mg1.jpg"> resolves to d:/assets/img/mg1.jpg -->
```

3. Inside `<BODY>` – headings & formatting

headings `<H1>` .. `<H6>`

```
<H1>BookYourHotel</H1>
<H2>Rating of the Hotels</H2>
<H6>Local Sight Seeing</H6>
```

BookYourHotel

Rating of the Hotels

Local Sight Seeing

☛ paragraphs & inline formatting

```
<P><B>Bold</B>, <I>italic</I>, <U>underline</U></P>  
<P>First line<BR>Second line after break</P>  
<HR> <!-- horizontal rule -->
```

Ordered list (numbered)

```
<OL>  
  <LI>Ranked 1st in hospitality</LI>  
  <LI>Ranked 2nd in infrastructure</LI>  
  <LI>Ranked 3rd in food variety</LI>  
</OL>
```

1. Ranked 1st in hospitality
2. Ranked 2nd in infrastructure
3. Ranked 3rd in food variety

ordered (default numbers)

Unordered list (bulleted)

```
<UL>  
  <LI>Deluxe rooms</LI>  
  <LI>Free breakfast</LI>  
  <LI>Sea view</LI>  
</UL>
```

- Deluxe rooms
- Free breakfast
- Sea view

unordered (disc bullets)

Difference between <DIV> and

<DIV>

Type	block-level element	inline element
Behaviour	starts on new line, takes full width	stays within the same line, only takes needed space
Common use	group large sections / layout blocks	style small portions of text / inline grouping

```
<DIV style="background:#d9eaff; padding:10px;">
  This is a DIV (block) – it creates a new block.
</DIV>
Just before <SPAN style="background: #ffe7b3; padding:3px;">this SPAN</SPAN> stays inline.
```

This is a DIV (block) – it creates a new block.

Just before **this SPAN** stays inline.

✓ DIV → block ; SPAN → inline

<DETAILS> and <SUMMARY> (interactive disclosure widget)

How it works: The `<SUMMARY>` element provides a visible heading that users can click. When clicked, it toggles the visibility of the additional content inside `<DETAILS>` – showing or hiding it without JavaScript.

```
<DETAILS>
  <SUMMARY>Categories of Rooms:</SUMMARY>
  <P>Standard, deluxe, suite ...</P>
</DETAILS>
```

► Categories of Rooms: ► click summary to expand/collapse

4. Adding images & multimedia

Image formats commonly used: GIF (drawings), JPEG (photos), PNG (high resolution).

with attributes

```
<IMG src="Room.jpg" alt="Deluxe Room" width="304" height="228">
```

<AUDIO>

- simple audio player

```
<AUDIO controls src="Audio1.mp3">  
  Your browser does not support audio.  
</AUDIO>
```

<VIDEO>

- with controls & autoplay

```
<VIDEO width="320" height="240" controls autoplay src="video.mp4">  
  Video fallback text.  
</VIDEO>
```

ATTRIBUTE	EXAMPLE	DESCRIPTION
autoplay	autoplay	starts as soon as ready
controls	controls	show play/pause/volume
loop	loop	restart when finished
muted	muted	audio output muted
poster	poster="preview.jpg"	image while video loads

5. Measuring data & progress bars

<METER>

- gauge (disk usage, scalar)


```
<METER value="4" min="0" max="10"></METER> <!-- 4 out of 10 -->
```



<PROGRESS> - task completion

```
Download: <PROGRESS value="12" max="100">12%</PROGRESS>
```



`max` = total work, `value` = completed portion.

6. Semantic tags (HTML5) - meaning & structure

TAG	PURPOSE
<code><ARTICLE></code>	self-contained content (blog post, news)
<code><ASIDE></code>	sidebar, note, tip (related to main)
<code><HEADER></code>	introductory heading / navigational block
<code><FOOTER></code>	footer for page/section (copyright, author)
<code><NAV></code>	navigation links
<code><FIGURE></code> + <code><FIGCAPTION></code>	image with caption

Example: figure with caption

```
<FIGURE>
  <FIGCAPTION> Deluxe Room</FIGCAPTION>
  <IMG src="Room.jpg" alt="deluxe" width="200">
</FIGURE>
```

🖼️ Deluxe Room [caption + image]

sample <ARTICLE> + <ASIDE>

```
<ARTICLE>
  <H3>BookYourHotel Blog</H3>
  <P>This blog provides hotel details.</P>
</ARTICLE>
<ASIDE>
  <P>Get discounts on booking rooms today!</P>
</ASIDE>
```

7. Navigation - <A> (anchor) tag

```
<A href="homepage.html" target="_blank">BookYourHotel</A>
```

`href` specifies the URL of a page or any website; `target` can open in a new window/frame (`"Frame1"`, `"_blank"`).

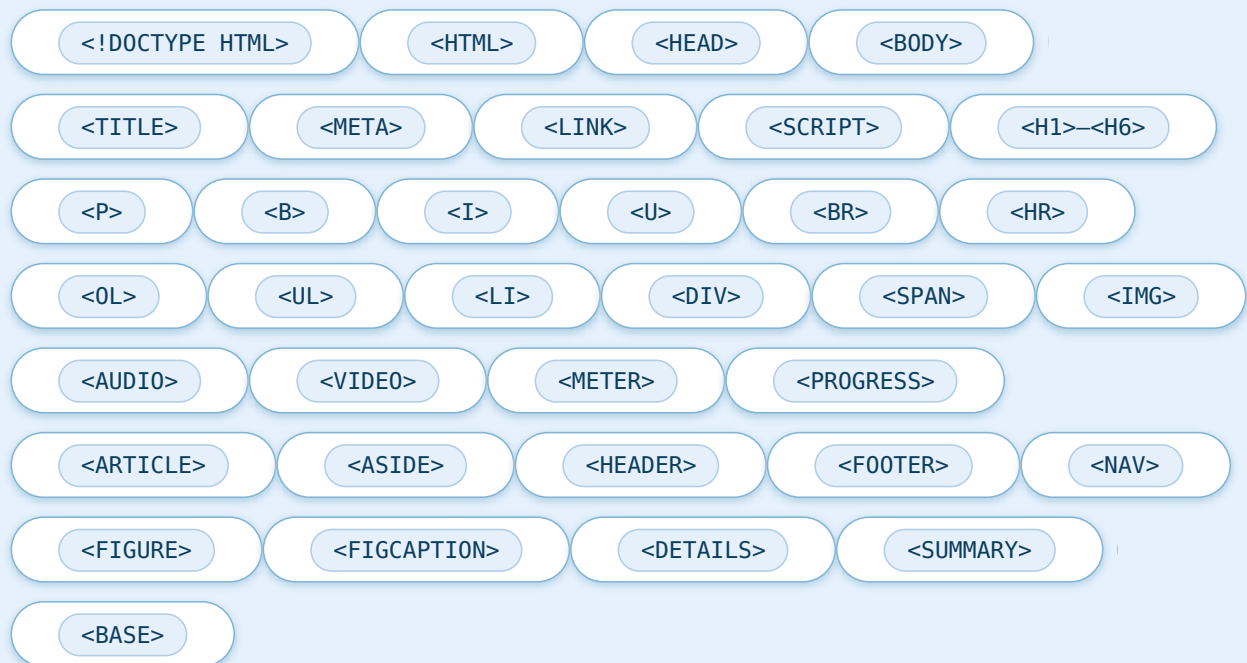
📌 8. Web page types & editors

- **Static web page:** fixed content.
- **Dynamic web page:** generated in real-time (e.g., news sites, Youku).

- **Text editor:** Notepad, VS Code – save with .htm or .html.
- **Graphic editor (WYSIWYG):** Dreamweaver – drag & drop elements.

The Internet is an interconnected network; the World Wide Web is a collection of interlinked pages.

Quick recap – tags covered in this chapter



✓ End of chapter 1 – from basic skeleton to semantic tags and multimedia.

Enhancing Web Pages

Chapter 2 · CSS – style sheets, selectors, properties, positioning

1. Introduction to CSS

cascading style sheets

CSS is a collection of styles that allows you to change the appearance of HTML elements on Web pages. It defines standard rules for layout and appearance, ensuring consistent design across a website.

Why CSS? Better control over page layout, enforce consistent display, separate content from presentation.

Syntax of a CSS rule

```
selector {  
  property: value;  
  /* multiple declarations */  
}
```

Example: make all <h1> red and italic, all <h2> bold.

```
h1 { color: red; font-style: italic; }  
h2 { font-weight: bold; }
```

result

Welcome
Subheading

2. Types of style sheets

Three ways to apply CSS: **inline**, **internal** (embedded), **external**.

Inline style

Applied directly to an element using the `style` attribute. Highest priority.

```
<P style="font-size:24pt; color:red;">Hotel booking from the comfort of your room.</P>
```

Hotel booking from the comfort of your room.

Internal (embedded) style sheet

Defined inside `<STYLE>` in the `<HEAD>`. Affects that page only.

```
<STYLE>
  p { color:red; font-size:20pt; font-style:italic; }
</STYLE>
```

External style sheet

Separate `.css` file linked via `<LINK>`. Best for multi-page consistency.

```
<LINK type="text/css" rel="stylesheet" href="externalstyle.css">
```

externalstyle.css content:

```
p { color:red; font-size:20pt; }
h1 { color:blue; font-weight:bold; }
```

Priority of style sheets (descending)

1. **Inline style** (highest)
2. **Internal style sheet**
3. **External style sheet**
4. **Browser default**

If same selector appears in multiple sheets, the most specific (highest priority) wins.

3. CSS selectors - ID and class

ID selector (#)

Used to style one unique element.

```
#pname { color:green; font-weight:bold; }
```

```
<P ID="pname">Welcome to BookYourHotel Website.</P>
```

Welcome to BookYourHotel Website.

Class selector (.)

Apply same style to multiple elements, possibly of different types.

```
.first { font-size:20px; color:red; font-weight:bold; }
```

```
<H1 class="first">Welcome</H1>  
<P class="first">This site gives info.</P>
```

➡ 4. Styling HTML elements

CSS properties are grouped into categories.

Font properties

PROPERTY	EXAMPLE VALUES
font-family	arial, helvetica, "times new roman"
font-size	12px, 100%, x-large
font-style	normal, italic, oblique
font-variant	small-caps
font-weight	bold, 600

```
p { font-family: "Helvetica"; font-size:20px; }
```

Text properties

PROPERTY	PURPOSE
text-align	left, right, center, justify
text-indent	25px, 5%
line-height	12pt, normal
color	red, #00ff00, rgb(0,0,255)
text-decoration	underline, overline, line-through
text-transform	uppercase, lowercase, capitalize
letter-spacing	2pt, -1px

List properties

PROPERTY	VALUES
list-style-type	disc, circle, square, decimal, lower-roman, upper-alpha ...

list-style-position	inside, outside
list-style-image	url('plus.gif')

```
ul { list-style-type:square; list-style-position:inside; }
```

Background properties

```
body { background-color: lavender; }  
div { background-image: url('img.gif'); }
```

5. Grouping and nesting styles

Grouping selectors

Apply same style to multiple selectors using comma.

```
h1, p { color: red; }
```

Nesting selectors (contextual)

SELECTOR	EXAMPLE	MEANING
element1 element2	div p	all <p> inside <div>
element1 > element2	div > p	<p> where parent is <div>
element1 + element2	div + p	<p> immediately after <div>
element1 ~ element2	div ~ p	<p> preceded by <div>

```
li a { color: red; text-decoration: none; } /* links inside list items */
```


6. Controlling visibility

Display property

```
display: none;    /* hidden, no space */  
display: block;   /* takes full width, line break */  
display: inline;  /* only necessary width, no line break */
```

Example: horizontal menu with `display: inline`

```
li { display: inline; list-style-type:none; }
```

Visibility property

```
visibility: hidden; /* hidden but space remains */  
visibility: visible; /* default */
```

visible

7. Positioning HTML elements

Position property

VALUE	DESCRIPTION
static	default, normal flow, no effect of top/left etc.

relative	relative to its normal position; space preserved
absolute	removed from flow; positioned relative to nearest positioned ancestor
fixed	relative to viewport; stays when scrolling

```
h3 { position: absolute; top:0px; }
```

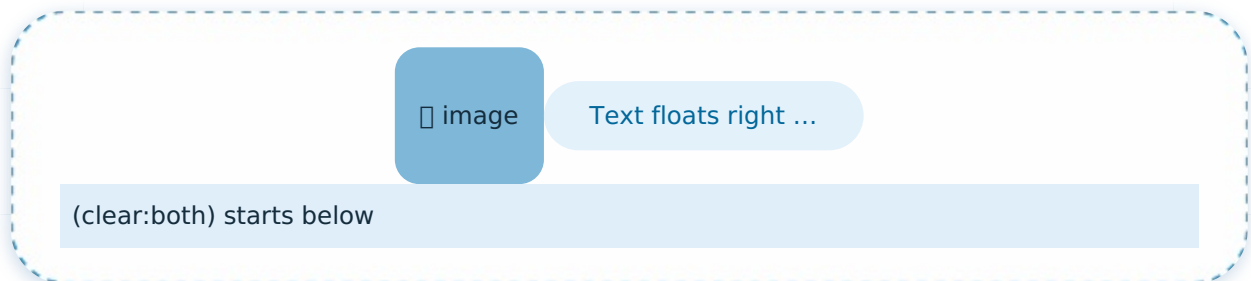
Use `top, bottom, left, right, z-index` with non-static.

Float & clear

```
float: left | right | none;
clear: both | left | right;
```

Float moves element to side, text wraps. **Clear** turns off float effect.

```
img { float: left; }
.special { clear: both; }
```



Quick recap – CSS properties & selectors

color

background-color

font-family

font-size

font-style

font-weight

font-variant

text-align

text-decoration

text-transform

letter-spacing

line-height

list-style

display

visibility

position

float

clear

top/left/right/bottom

z-index

id (#)

class (.)

:link

:visited

:hover

:active

✓ Chapter 2 summary - from inline styles to positioning & floats.

✂ Enhancing Web Pages · complete CSS notes

Working with Tables and Frames

Chapter 3 · HTML tables, `<IFRAME>`, colspan, rowspan, styling

1. Why tables? Structured information

Tables present complex data in rows and columns, making it readable. Example: LearnMySQL data types shown as table vs. plain text.

Char(M) Fixed 0-255
Varchar(M) Variable 0-255

NAME	LENGTH	RANGE
Char(M)	Fixed	0-255
Varchar(M)	Variable	0-255

core tag

Tables are created with `<TABLE>`. Use `border` attribute to show borders.

2. Basic structure of a table

A table can be split into **header** (`<THEAD>`), **body** (`<TBODY>`), and **footer** (`<TFOOT>`). Caption gives a title.

Minimal table with rows & columns

```
<TABLE border="1">
  <TR><TD>Name</TD><TD>Login ID</TD></TR>
  <TR><TD>Steve</TD><TD>01234</TD></TR>
</TABLE>
```

Name	Login ID
Steve	01234

simple table

Using `<THEAD>`, `<TBODY>`, `<TFOOT>` (order: THEAD, TFOOT, then TBODY)

```
<TABLE border="1">
  <CAPTION>Test Results</CAPTION>
  <THEAD>
    <TR><TH>Name</TH><TH>Marks</TH></TR>
  </THEAD>
  <TFOOT>
    <TR><TD colspan="2">Total: 3 students</TD></TR>
  </TFOOT>
  <TBODY>
    <TR><TD>Joseph</TD><TD>98</TD></TR>
    <TR><TD>Steve</TD><TD>92</TD></TR>
  </TBODY>
</TABLE>
```

`<TH>` vs `<TD>`

`<TH>` creates header cells (bold, centered), `<TD>` standard data cells.

3. Merging cells: colspan & rowspan

attributes `colspan` merges columns, `rowspan` merges rows.

→ colspan example

```
<TR><TD colspan="3">Top Three Performers</TD></TR>
```

Top Three Performers		
Name	Login	Marks

↓ rowspan example

```
<TR><TD rowspan="3">Top Three</TD><TD>Joseph</TD></TR>
<TR><TD>Steve</TD></TR>
<TR><TD>John</TD></TR>
```

Top Three	Joseph
	Steve
	John

4. Styling tables with CSS

CSS properties to improve table appearance:

PROPERTY	EFFECT	EXAMPLE
border	sets border width/style/color	border: 1px solid brown;
border-collapse	collapse/separate borders	border-collapse: collapse;
padding	space between content and border	padding: 10px;
text-align	alignment inside cells	text-align: center;
color	text color	color: blue;
background-color	background of cells/table	background-color: #BDB76B;
width / height	table dimensions	width: 100%; height: 200px;

```
<STYLE>
  th { background-color: #BDB76B; color: #006400; padding: 5px; }
  td { border: 1px solid #008B8B; padding: 3px; }
  caption { font-size: 35px; }
</STYLE>
```

5. Frames – displaying multiple pages

A frame is a rectangular region in a browser window that displays a separate Web page. In modern HTML we use `<IFRAME>` (inline frame).

Example: LearnMySQL site with header, navigation, content frames.

`<IFRAME>` basic attributes

ATTRIBUTE	PURPOSE
<code>src</code>	URL of the page to embed
<code>name</code>	target name for links
<code>height / width</code>	size of the iframe
<code>seamless</code>	boolean – frame appears part of page (no borders/scrollbars)
<code>srcdoc</code>	inline HTML to display (overrides <code>src</code>)

Creating three frames (home.html)

```
<IFRAME name="HeaderFrame" width="91%" height="100" srcdoc="<h1>Learn MySQL</h1>">
</IFRAME>
<IFRAME src="nav.html" name="NavigationFrame" width="25%" height="500"></IFRAME>
<IFRAME name="ContentFrame" width="65%" height="500"></IFRAME>
```

Targeting a frame (anchor target)

Make links open inside a specific iframe using `target="frameName"`.

```
<A href="courses.html" target="ContentFrame">Courses</A>
```

↪ Click → opens in "ContentFrame"

6. Styling frames with CSS

PROPERTY	EFFECT
margin	space outside the iframe
padding	space between border and content
border	border thickness, style, color
scrolling	yes/no/auto (not CSS, but an attribute; can be styled via overflow in CSS)

```
iframe {  
  border: 5px solid black;  
  margin-left: 1px;  
  padding: 2px;  
}
```

Quick recap – chapter 3 tags & CSS

<TABLE>	<TR>	<TD>	<TH>	<THEAD>
<TBODY>	<TFooter>	<CAPTION>	colspan	rowspan
<IFRAME>	src	name	target	seamless
srcdoc	border	border-collapse	padding	
margin				

Tables for structured data · Frames for multi-page views · CSS for polished look.

Creating Dynamic Web Pages

Chapter 4 · HTML forms, input elements, JavaScript basics, form objects

1. Forms – accepting user input

<FORM>

Forms are interactive elements that collect data from users. Example: OrderFoodOnline form.

Scenario: FoodExpress.com / BookYourHotel – customers fill details like name, contact, preferences.

Basic form structure

```
<FORM name="OrderFoodOnline" method="post" action="process.html">
  ... form elements ...
</FORM>
```

📌 Important form attributes

ATTRIBUTE	DESCRIPTION
name	unique name for the form (used in scripts)
id	unique identifier for CSS/JavaScript
action	URL where form data is sent
method	"get" (appends data to URL, visible, limited) or "post" (secure, no size limit)
autocomplete	"on" / "off" – browser autofill
novalidate	disable browser validation
target	where to show response (_blank, _self, _parent, _top, frame_name)

2. <INPUT> tag and its types

The most versatile element – `type` attribute defines behaviour.

Text / Password / Email / Tel / URL / Number

TYPE	CODE EXAMPLE
text	<code><INPUT type="text" name="fname" placeholder="First name"></code>
password	<code><INPUT type="password" name="pwd"></code>
email	<code><INPUT type="email" name="email" multiple></code> (auto-validate)
tel	<code><INPUT type="tel" name="phone"></code>
url	<code><INPUT type="url" name="website"></code>
number	<code><INPUT type="number" min="0" max="50"></code>

Radio & Checkbox

```
<INPUT type="radio" name="food" value="veg" checked> Vegetarian
<INPUT type="radio" name="food" value="nonveg"> Non-vegetarian
<INPUT type="checkbox" name="cuisine1" value="Continental"> Continental
```

Date / Time / Range

```
<INPUT type="date" name="bday">
<INPUT type="time" name="usr_time">
<INPUT type="range" min="10" max="50" step="5" value="10">
```

🖼 Image as submit button

```
<INPUT type="image" src="submit.gif" alt="submit" width="48" height="48">
```

Common attributes for <INPUT>

- `name` – identifies field
- `id` – unique for CSS/JS
- `value` – default / button text
- `placeholder` – hint inside field
- `required` – must be filled
- `autofocus` – focus on load
- `pattern` – regex validation
- `autocomplete` – on/off
- `disabled` / `readonly`
- `maxlength`, `size`

example placeholder & required

Enter your name

3. Drop-down lists: <SELECT>, <OPTION>, <OPTGROUP>

```
<SELECT name="hotels" size="1" multiple>
  <OPTION value="h1" selected>Taj</OPTION>
  <OPTION value="h2">Oberoi</OPTION>
</SELECT>
```

`size` >1 shows list box; `multiple` allows Ctrl+select.

☒ Grouping with <OPTGROUP>

```
<SELECT>
  <OPTGROUP label="Italian">
    <OPTION value="pasta">Pasta</OPTION>
    <OPTION value="fish">Fish</OPTION>
  </OPTGROUP>
  <OPTGROUP label="Chinese">
    <OPTION value="chow">Chowmein</OPTION>
  </OPTGROUP>
</SELECT>
```

4. Labels and grouping

<LABEL> – improves accessibility

```
<LABEL for="name">Name:</LABEL>  
<INPUT type="text" id="name">
```

Clicking the label focuses the input.

<FIELDSET> and <LEGEND>

```
<FIELDSET>  
  <LEGEND>Food preferences</LEGEND>  
  <!-- grouped fields -->  
</FIELDSET>
```

The diagram shows a dashed rounded rectangle representing a fieldset. Inside, there is a light blue rounded rectangle representing a legend with the text "Food". Below the legend, there is a light blue rounded rectangle representing a group of fields with the text "Italian, Chinese ...".

5. Other form elements

<TEXTAREA> – multi-line text

```
<TEXTAREA rows="3" cols="16">Enter address</TEXTAREA>
```

<DATAList> – autocomplete suggestions

```
<INPUT list="states">
<DATAList id="states">
  <OPTION value="Alabama">
  <OPTION value="California">
</DATAList>
```

<KEYGEN> (deprecated) – key-pair generation

```
<KEYGEN name="key1" keytype="rsa">
```

<OUTPUT> – calculation result

```
<FORM oninput="res.value=parseInt(a.value)+parseInt(b.value)">
  <INPUT type="number" id="a"> + <INPUT type="number" id="b">
  = <OUTPUT name="res" for="a b"></OUTPUT>
</FORM>
```

<BUTTON> – versatile button

```
<BUTTON type="submit"><IMG src="icon.png"> Send</BUTTON>
<BUTTON type="reset">Reset</BUTTON>
```

⚡ 6. JavaScript basics – making pages dynamic

JavaScript is an object-based scripting language that runs in the browser (and elsewhere). It can manipulate page elements, react to events, and validate forms.

Adding JavaScript to HTML

```
<!-- internal script -->
<SCRIPT>
  alert('Hello, world!');
</SCRIPT>

<!-- external script -->
<SCRIPT src="myscript.js"></SCRIPT>
```

Variables & data types

```
let message = "Hello";    // modern
var name = "John";        // older
const PI = 3.14;
age = 25;                  // not recommended
```

Operators

CATEGORY	EXAMPLES
Arithmetic	+ - * / %
Assignment	= += -= *= /= %=
Comparison	< > <= >= == != ===
Logical	&& !

7. Functions – reusable blocks

```
function showMessage(from, text) {
  alert(from + ': ' + text);
}
showMessage('Ann', 'Hello');
```

Functions can return values:

```
function sum(a, b) {  
  return a + b;  
}  
let result = sum(5, 3); // 8
```

Parameters vs arguments: Parameters are declared in the function; arguments are passed when called.

8. Form objects & validation

JavaScript represents each form and its elements as objects you can access. The recommended modern way to access individual fields is using `document.getElementById()`.

Accessing form fields with `getElementById`

```
// Access by unique ID (recommended)  
let nameField = document.getElementById("username");  
let nameValue = nameField.value;  
  
// For forms, you can also get the form then its elements  
let myForm = document.getElementById("orderForm");  
let emailField = myForm.elements["email"]; // or use getElementById on each field
```

Best practice: assign an `id` to each input field and access it directly with `document.getElementById("fieldId")`. This is cleaner and more reliable than navigating through forms array.

Simple validation example using `getElementById`

```
<SCRIPT>  
function validateForm() {  
  let fname = document.getElementById("fname").value;  
  if (fname == "") {
```



```

    alert("Name must be filled out");
    return false;
  }
  return true;
}
</SCRIPT>
<FORM name="myForm" onsubmit="return validateForm()">
  Name: <INPUT type="text" id="fname" name="fname">
  <INPUT type="submit" value="Submit">
</FORM>

```

Common form object methods & events

OBJECT	METHODS	EVENTS
form	submit(), reset()	onsubmit, onreset
button	click(), focus(), blur()	onclick, onmousedown
checkbox / radio	click(), focus()	onclick, onchange
text / textarea	select(), focus(), blur()	onfocus, onchange, onselect
select	focus(), blur()	onchange

Quick recap – chapter 4 tags & concepts

<FORM>

action

method

<INPUT>

type

placeholder

required

pattern

<SELECT>

<OPTION>

<OPTGROUP>

<LABEL>

<FIELDSET>

<LEGEND>

<TEXTAREA>

<DATALIST>

<BUTTON>

<OUTPUT>

JavaScript

variables

function

onsubmit

form objects

getElementById

value property

From form structure to client-side scripting – dynamic pages. Use `getElementById` to access form fields directly.

✂ Creating Dynamic Web Pages · complete notes · forms, input types, JavaScript, `getElementById`

Adding Visual Effects to Web Pages

Chapter 5 · jQuery library, selectors, events, effects (hide/show, slide, fade, animate), image gallery (colorbox)

1. What is jQuery?

lightweight JS library

jQuery is an open-source, cross-browser JavaScript library that simplifies HTML manipulation, event handling, and animation. It wraps complex JavaScript into one-line methods.

Why jQuery? Write less, do more. Abstracts browser inconsistencies, provides powerful selectors, and built-in effects.

Including jQuery in a Web Page

```
<SCRIPT src="jquery-1.8.3.js"></SCRIPT>
```

You can download the library or use a CDN. The code must be placed after the jQuery file.

document.ready() – ensure page is loaded

```
$(document).ready(function(){  
    // jQuery code executes after DOM is fully loaded  
});
```

Prevents errors if script runs before elements exist

2. jQuery selectors

jQuery uses CSS-like selectors to target elements. Basic syntax: `$(selector).action()`

By element name, ID, class

```
$("#h1")           // all <h1> elements
$("#header")       // element with id="header"
$(".product")       // all elements with class="product"
$("#h1#main")       // <h1> with id="main"
$("div.product")    // <div> with class="product"
```

By attribute

```
$("[src]")          // elements with src attribute
$("[src='movie.jpg']") // specific attribute value
```

Nesting / hierarchy

```
$("#menu ul")       // all <ul> inside element with id="menu"
$("#menu li a")      // links inside list items inside menu
```

Example: Hide all submenus on button click – `$('#menu ul').hide();`

3. Manipulating content

| METHOD | DESCRIPTION |
|-------------------------------|--|
| <code>html(content)</code> | Sets or returns HTML content |
| <code>append(content)</code> | Inserts content at the end of selected element |
| <code>prepend(content)</code> | Inserts content at the beginning |
| <code>after(content)</code> | Inserts content after selected element |

| | |
|-----------------------|---|
| <code>remove()</code> | Removes selected element and its children |
| <code>empty()</code> | Removes child elements only |

```
$("#para").prepend("<b>jQuery</b> ");
$("#para").append(" <b>append() function</b>");
$("#newhead").remove();
```

⚡ 4. Handling events with jQuery

Event methods attach functions to user interactions.

| EVENT METHOD | FIRED WHEN... |
|-------------------------------|----------------------|
| <code>click()</code> | element clicked |
| <code>dblclick()</code> | double-click |
| <code>mouseenter()</code> | mouse enters element |
| <code>mouseleave()</code> | mouse leaves element |
| <code>keydown()</code> | key pressed |
| <code>submit()</code> | form submitted |
| <code>focus() / blur()</code> | focus in/out |

```
$("#button").click(function(){
  $("#h1").hide();
});
```

Example: click → hide header

Hide demo header

This is a demo header

5. Visual effects: hide / show / toggle / slide / fade

/ animate

Hide & Show

```
$("#h1").hide("slow");  
$("#h1").show(2000);    // milliseconds
```

Toggle (alternate hide/show)

```
$("#p").toggle("fast");
```

Slide effects

| METHOD | EFFECT |
|----------------------------|-----------------------------------|
| <code>slideDown()</code> | reveals with sliding motion |
| <code>slideUp()</code> | hides with sliding up |
| <code>slideToggle()</code> | toggles between slideUp/slideDown |

```
$(".flipbut").click(function(){  
    $(".thought").slideToggle();  
});
```

☒ Fade effects

```
$("#div").fadeOut(1200);    // fade out  
$("#button").click(function(){  
    $("#div").fadeIn(1200);    // fade in  
});
```

Custom animate()

```
$("#div").animate({  
  left: '250px',  
  opacity: '0.5',  
  height: '250px',  
  width: '250px'  
}, 1000);
```

animate() syntax: `animate({params}, speed, callback);` – changes CSS properties smoothly.

6. Real-world example – sliding product menu

ShopForYou.com: product categories (Computers, Entertainment, Cameras) with sub-items that slide toggle on click.

```
$(document).ready(function(){  
  $("#menu ul").hide(); // initially hide submenus  
  $("#menu li a").click(function(){  
    $(this).next().slideToggle('slow');  
  });  
});
```

Click on "Laptops" → submenu slides down, click again → slides up

7. Image gallery using colorbox plugin

jQuery plugins extend functionality. **Colorbox** is a lightweight lightbox plugin for image galleries and slideshows.

Setup steps (summary)

- Download colorbox.zip and extract.
- Include `colorbox.css` and `jquery.colorbox.js`.
- Call `$("#a").colorbox();`

Basic usage – single image

```
<LINK rel="stylesheet" href="css/colorbox.css">
<SCRIPT src="jquery-1.8.3.js"></SCRIPT>
<SCRIPT src="jquery.colorbox.js"></SCRIPT>
<SCRIPT>
  $(document).ready(function(){
    $("#a").colorbox();
  });
</SCRIPT>
<A href="product.png">View product</A>
```

☒ Customizing colorbox (transition, size)

```
$("#a").colorbox({
  transition: "fade",
  height: "250",
  width: "220"
});
```

☒ Group of images (gallery with prev/next)

```
<A rel="gallery" href="img1.jpg">Image 1</A>
<A rel="gallery" href="img2.jpg"></A>
<A rel="gallery" href="img3.jpg"></A>
```

Use same `rel` value to group images. Navigation arrows appear automatically.

☒ Slideshow mode


```
$("#a").colorbox({slideshow: true, current: false});
```

8. galleria plugin – professional galleries

Another popular jQuery plugin that automatically creates thumbnails and displays larger versions on hover. Ideal for image-heavy websites.

galleria vs colorbox: colorbox creates lightbox overlays, galleria builds thumbnail-based galleries inside the page.

Quick recap – chapter 5 concepts

jQuery library

`$(document).ready()`

Selectors (`#id`, `.class`, `element`)

`append()` / `prepend()`

`remove()`

`click()`

`hide()` / `show()` / `toggle()`

`slideDown()` / `slideUp()` / `slideToggle()`

`fadeOut()` / `fadeIn()`

`animate()`

colorbox plugin

`rel="gallery"`

slideshow

galleria plugin

jQuery simplifies DOM manipulation, events, and animations. Plugins like colorbox create stunning galleries.

Complete Bootstrap 4.1.3 Notes

1. Introduction to Bootstrap

What is Bootstrap? – A free, open-source CSS framework developed by Twitter. It provides responsive grid system, UI components, and JavaScript plugins. Saves time with consistent design across browsers.

Key benefits & philosophy

- **Mobile-first:** styles are built for small screens then scale up.
- **12-column flexbox grid:** easy responsive layouts.
- **Extensive components:** navbar, cards, dropdowns, modals, forms.
- **Customizable:** override with Sass variables or custom CSS.

Important setup classes & CDN

.container

.container-fluid

meta viewport

bootstrap.css

jQuery slim

popper.js

bootstrap.js

```
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
<link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.1.3/css/bootstrap.min.css">
<script src="https://code.jquery.com/jquery-3.3.1.slim.min.js"></script>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.14.3/umd/popper.min.js">
</script>
<script src="https://stackpath.bootstrapcdn.com/bootstrap/4.1.3/js/bootstrap.min.js">
</script>
```

🖥️ LIVE NOTE:

The viewport meta ensures proper responsive scaling on all devices. Bootstrap JS requires jQuery & Popper for dropdowns, modals, toggles.

2. Grid System

Bootstrap grid is built with flexbox. It uses 12 columns, rows (`.row`) and columns (`.col-*-*`). Responsive breakpoints control when columns stack horizontally.

Responsive breakpoints & important classes

Class prefix	Breakpoint	Description
<code>.col-</code>	<576px	Extra small devices (portrait phones) – no media query, always horizontal
<code>.col-sm-</code>	≥576px	Small devices (landscape phones)
<code>.col-md-</code>	≥768px	Medium devices (tablets)
<code>.col-lg-</code>	≥992px	Large devices (laptops/desktops)
<code>.col-xl-</code>	≥1200px	Extra large desktops

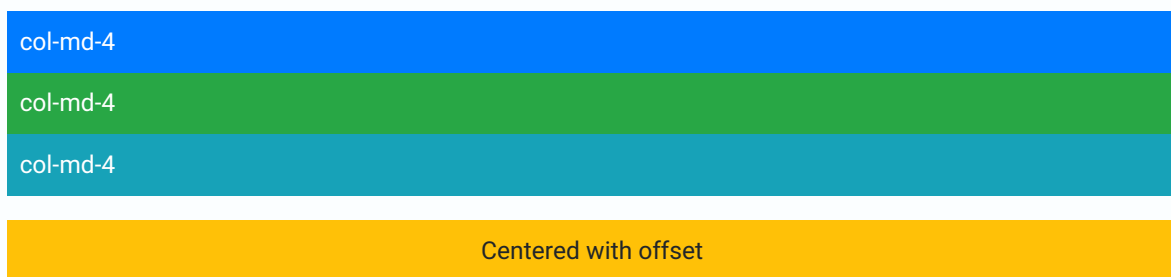
Important grid classes explained

- `.row` – flex container, negative margins
- `.col` – auto-width column
- `.col-{breakpoint}-{number}` – defines column count (1-12)
- `.offset-{breakpoint}-{number}` – pushes columns right
- `.order-{breakpoint}-{number}` – reorder columns visually
- `.no-gutters` – removes column padding

Code & exact output matching

```
<div class="row">
  <div class="col-md-4 bg-primary text-white p-2">col-md-4</div>
  <div class="col-md-4 bg-success text-white p-2">col-md-4</div>
  <div class="col-md-4 bg-info text-white p-2">col-md-4</div>
</div>
<div class="row mt-3">
  <div class="col-md-6 offset-md-3 bg-warning p-2 text-center">Centered with
  offset</div>
</div>
```

👁️ OUTPUT:



3. Typography & Text Utilities

Bootstrap sets global defaults: font-family, line-height, and heading styles. Use `.display-*` for prominent headings, `.lead` for emphasis.

Important typography classes

- `.display-1` to `.display-4` – larger, lighter headings
- `.lead` – makes paragraph stand out (font-size 1.25rem)
- `.text-muted` – muted gray text

`.text-primary/.success/.danger/.warning`

– contextual colors

`.text-uppercase/.capitalize`

– text transformation

`.font-weight-bold/.italic`

– font styles

```
<h1 class="display-4">Display heading</h1>
<p class="lead">This paragraph uses .lead for emphasis.</p>
<p class="text-primary font-weight-bold">Primary bold text</p>
<p class="text-danger text-uppercase">danger uppercase alert</p>
```

👁️OUTPUT:

Display heading

This paragraph uses .lead for emphasis.

Primary bold text

DANGER UPPERCASE ALERT

4. Forms & Form Controls

Bootstrap provides global form styling. `.form-control` gives full-width, rounded corners, and focus glow. `.form-group` adds bottom margin. For inline forms use `.form-inline`.

Important form classes

`.form-control` – styles input, select, textarea

`.form-control-lg/.sm` – sizing variants

`.form-group`

– spacing between form items

`.form-check / .form-check-input`

– checkboxes/radios

`.form-inline`

– inline form (horizontal compact)

```
<div class="form-group">
  <label>Email</label>
  <input type="email" class="form-control" placeholder="name@example.com">
</div>
<div class="form-check">
  <input class="form-check-input" type="checkbox">
  <label class="form-check-label">Remember me</label>
</div>
<button class="btn btn-primary">Submit</button>
<form class="form-inline mt-3">
  <input class="form-control mr-2" placeholder="Username">
  <button class="btn btn-success">Login</button>
</form>
```

👁️OUTPUT:

Email

name@example.com

☐ Remember me

Submit

Username

Login

5. Buttons, Button Groups & Badges

✂ Buttons use `.btn` base class + modifier `.btn-primary`, `.btn-outline-*`. Sizes: `.btn-lg`, `.btn-sm`. Group with `.btn-group`.

All important button classes

`.btn-primary`, `.btn-secondary`, `.btn-success`, `.btn-danger`

`.btn-outline-light`, `.btn-outline-dark`

`.btn-lg`, `.btn-sm`, `.btn-block`

`.btn-group`, `.btn-group-vertical`

`.badge`, `.badge-pill`, `.badge-primary`

```
<button class="btn btn-primary">Primary</button>
<button class="btn btn-outline-danger">Outline Danger</button>
<button class="btn btn-success btn-sm">Small Success</button>
<div class="btn-group mt-2">
  <button class="btn btn-secondary">Left</button>
  <button class="btn btn-secondary">Middle</button>
</div>
<h3>Alerts <span class="badge badge-warning">3</span></h3>
```

👁 OUTPUT:

Primary

Outline Danger

Small Success

Left

Middle

Alerts 3

▼ 6. Dropdowns

Dropdowns require Bootstrap JS + Popper. Use `.dropdown` wrapper, `data-toggle="dropdown"` on the trigger, and `.dropdown-menu` for the menu.

Important dropdown classes

`.dropdown`

`.dropdown-toggle`

`.dropdown-menu`

`.dropdown-item`

`.dropdown-divider`

`.dropup`

`.dropright`

```

<div class="dropdown">
  <button class="btn btn-secondary dropdown-toggle" type="button" data-
toggle="dropdown">
    Dropdown
  </button>
  <div class="dropdown-menu">
    <a class="dropdown-item" href="#">Profile</a>
    <a class="dropdown-item" href="#">Settings</a>
    <div class="dropdown-divider"></div>
    <a class="dropdown-item" href="#">Logout</a>
  </div>
</div>

```

🔗 **LIVE INTERACTIVE OUTPUT (CLICK):**

Dropdown ▾

7. List Group & Input Group

List group: displays series of content. Can be used with badges, custom content. **Input group:** extends form controls with add-ons (text, buttons, icons).

Important classes

.list-group

.list-group-item

.list-group-item-action

.list-group-flush

.input-group

.input-group-prepend

.input-group-append

.input-group-text

```

<ul class="list-group">
  <li class="list-group-item active">Inbox <span class="badge badge-light">12</span>
</li>
  <li class="list-group-item">Sent</li>
</ul>
<div class="input-group mt-3">
  <div class="input-group-prepend"><span class="input-group-text">@</span></div>
  <input type="text" class="form-control" placeholder="username">
</div>

```

🔗 **OUTPUT:**

Inbox 12

Sent

@ username

8. Navigation & Responsive Navbar

Navbar is responsive and collapsible. `.navbar-expand-{breakpoint}` defines when navbar expands horizontally. `.navbar-dark/.navbar-light` for text colors.

Important navbar classes

`.navbar``.navbar-brand``.navbar-nav``.nav-item``.nav-link``.navbar-toggler``.collapse .navbar-collapse``.ml-auto .mr-auto`

```
<nav class="navbar navbar-expand-md navbar-dark bg-dark">
  <a class="navbar-brand" href="#">MyApp</a>
  <button class="navbar-toggler" data-toggle="collapse" data-target="#demoNav">
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="demoNav">
    <ul class="navbar-nav ml-auto">
      <li class="nav-item active"><a class="nav-link" href="#">Home</a></li>
      <li class="nav-item"><a class="nav-link" href="#">Features</a></li>
    </ul>
  </div>
</nav>
```

[👁 LIVE DEMO \(RESIZE TO SEE COLLAPSE\):](#)

9. Icons (Font Awesome integration)

Bootstrap 4 doesn't have built-in icons. Use Font Awesome CDN for scalable vector icons. Add `<i class="fas fa-[name]"></i>`.

```
<i class="fas fa-home"></i> Home
<i class="fas fa-heart text-danger"></i> Like
<button class="btn btn-primary"><i class="fas fa-save"></i> Save</button>
```

[👁 OUTPUT:](#)

[!\[\]\(683dba75afe26e28cd4de5730b776760_img.jpg\) Home](#) [!\[\]\(04de6559f804bf16e9c902ae57940aeb_img.jpg\) Like](#) [!\[\]\(e27fb107ee3507541090ecc7198535b4_img.jpg\) Save](#)

10. Breadcrumb Navigation

Breadcrumbs indicate current page location. Uses `.breadcrumb` and `.breadcrumb-item`.

```
<nav>
  <ol class="breadcrumb">
    <li class="breadcrumb-item"><a href="#">Home</a></li>
    <li class="breadcrumb-item active">Bootstrap</li>
  </ol>
</nav>
```

👁️ OUTPUT:

[Home](#) / Bootstrap

11. Spacing Utilities (Margin & Padding)

Shorthand: `{property}{sides}-{size}`.

Property: `m` (margin), `p` (padding). Sides: t, b, l, r, x, y. Sizes: 0–5, auto.

Example: `.mt-3` (margin-top 1rem), `.px-4` (padding-x 1.5rem), `.mx-auto` (horizontal center)

`.m-1 to .m-5`

`.mt-2 .mb-3 .ml-auto .mx-auto`

`.p-3 .pt-4 .px-2`

`.d-flex .justify-content-center`

```
<div class="bg-light p-4 mb-3">Large padding + margin bottom</div>
<div class="bg-warning p-2 mx-auto text-center" style="max-width: 250px;">Centered
block</div>
```

👁️ OUTPUT:

Large padding + margin bottom

Centered block

Chapter 12 - Introduction to VueJS

1. What is VueJS?

VueJS (pronounced "view") is an open-source progressive JavaScript framework used to develop interactive web interfaces. It is one of the most famous frameworks used to simplify web development.

- **Progressive Framework:** You can start small and scale up as needed
- **Approachable:** If you know HTML, CSS & JavaScript, you're ready to develop Vue apps in no time
- **Versatile:** Can be embedded into any web app or existing project
- **Lightweight:** Only 20KB min+gzipped runtime blazing fast framework
- **Perfect for SPAs:** Ideal for building Single Page Applications

Key Point: Vue focuses on the **view layer** only, making it easy to integrate with other libraries or existing projects.

⚡ 2. Features of VueJS

□ Virtual DOM

A "lite" version of HTML DOM that uses diff algorithms to avoid re-rendering the entire DOM after changes.

Data Binding

Manipulate HTML attributes, change style, and assign classes using the `v-bind` directive.

Components

Create custom, reusable elements that can be used in HTML.

Event Handling

`v-on` attribute listens to DOM events in VueJS.

Animation/Transition

Built-in transition component with third-party animation library support.

Computed Properties

Listen to UI changes and perform calculations without additional coding.

Templates

HTML-based templates that bind DOM with Vue instance data.

⚙ Directives

Special attributes like `v-if`, `v-for`, `v-model` to manipulate DOM.

📦 Vue-CLI

Command Line Interface to build and compile projects easily.

3. Understanding DOM & Virtual DOM

What is DOM (Document Object Model)?

The Document Object Model, or DOM, is an interface that treats all markup language (your HTML) as connected nodes. It can be seen as an interface of objects for markup elements stored in a tree-like structure.

The DOM allows us to write and change styles on elements, and even change elements themselves. This is done by adding, modifying, or deleting element tags or CSS styles in the head and body of a document because they are represented as nodes and objects.

⚠ Disadvantages of Traditional DOM

As applications grow, more nodes to traverse, more elements, and more scripts to communicate with those elements — the DOM grows slower and costs a lot of processing power. Trying to perform a search or even update a DOM becomes a hard task, ultimately affecting the performance of the application.

How Vue.js Virtual DOM Works

Step 1 Initial Render

Vue creates a Virtual DOM from the template and renders it to the real DOM.

```
Template
<div id="app">
  <h1>{{ title }}</h1>
  <p>{{ message }}</p>
  <button @click="update">Update</button>
</div>
```



```
Virtual DOM (in memory)
• div#app
• h1 → "Hello Vue!"
• p → "This is Vue.js"
• button → "Update"
```

Step 2 Data Changes

When data changes, Vue creates a new Virtual DOM (New VDOM).

```
Data Change
// change
```

```
this.message = 'Virtual DOM is awesome!'
```

```
New Virtual DOM (in memory)
<div id="app">
  <h1>Hello Vue!</h1>
  <p>Virtual DOM is awesome!</p>
  <button>Update</button>
</div>
```

Step 3 Diffing (Compare)

Vue compares the New VDOM with the Old VDOM to find the minimal changes.

```
Old Virtual DOM
<div id="app">
  <h1>Hello Vue!</h1>
  <p>This is Vue.js</p>
  <button>Update</button>
</div>
```



```
New Virtual DOM
<div id="app">
  <h1>Hello Vue!</h1>
  <p>Virtual DOM is awesome!</p>
  <button>Update</button>
</div>
```

Diff Result: Only the `<p>` tag content changed from "This is Vue.js" to "Virtual DOM is awesome!"

Step 4 Patch

Vue creates a patch (instructions) with only the necessary changes.

Patch (instructions):

- Set `<p>` text to 'Virtual DOM is awesome!'

Step 5 Apply Patch to Real DOM

Vue applies the patch to the real DOM. Only the changed part is updated.

```
Real DOM (in browser)
<div id="app">
  <h1>Hello Vue!</h1>
  <p>Virtual DOM is awesome!</p>
  <button>Update</button>
</div>
```

Step 6 DOM is in Sync

The real DOM is now in sync with the latest data. UI is updated with minimal operations.

```
Final Real DOM (in browser)
<div id="app">
  <h1>Hello Vue!</h1>
  <p>Virtual DOM is awesome!</p>
  <button>Update</button>
</div>
```

Benefits of Virtual DOM:

- Better Performance
- Minimal DOM operations
- Faster UI updates
- More efficient

Summary: Vue uses Virtual DOM as an intermediary layer. When data changes, it updates the Virtual DOM, compares it with the previous one (diffing), creates a patch, and applies only the necessary changes to the Real DOM.

4. VueJS Installation - Complete Setup

Prerequisite: Download and install [Node.js](#) from the official website.

Step 1: Verify Node.js and npm Installation

```
node -v
npm -v
```

Step 2: Install Vue CLI Globally

```
npm install -g @vue/cli
```

Step 3: Verify Vue CLI Installation

```
vue --version
```

Step 4: Create a New Vue 3 Project

```
vue create my-vue-project
```

When prompted:

- Choose **"Manually select features"**
- Select: **Babel, Vuex, Router, Linter**
- Choose **Vue 3.x** version

Step 5: Navigate to Project and Run Server

```
cd my-vue-project
npm run serve
```

Alternative Method (Vite - Faster):

```
npm create vue@latest my-project
```

Then: `cd my-project && npm install && npm run dev`

5. Vue CLI Project Structure

```
my-vue-project/
├── node_modules/  ← All project dependencies
├── public/        ← Static assets
├── index.html     ← Main HTML file
├── src/          ← Source code
│   ├── assets/   ← Images, CSS files
│   ├── components/ ← Reusable Vue components
│   ├── router/   ← Vue Router configuration
│   ├── store/    ← Vuex state management
│   └── views/    ← Page-level components
├── App.vue       ← Root component
├── main.js       ← Application entry point
├── package.json  ← Dependencies and scripts
└── README.md     ← Project documentation
```

.vue File Structure (Single File Component)

```
<template>
  <div>HTML / UI part</div>
</template>

<script>
export default {
  name: 'ComponentName',
  data() { return { ... } },
  methods: { ... }
}
</script>

<style scoped>
/* CSS styles - only for this component */
</style>
```

6. Recommended VS Code Extensions

- **Vue VSCode Snippets** – Code snippets for faster development
- **Prettier** – Code formatter

7. Useful Vue CLI Commands

```
# Create a new project
vue create project-name

# Run development server
npm run serve

# Build for production
npm run build
```

Chapter Summary

In this chapter, you have learned:

- VueJS is a progressive framework for building user interfaces
- Features of VueJS: Virtual DOM, Data Binding, Components, Event Handling, Animation/Transition, Computed Properties, Templates, Directives, Vue-CLI
- Virtual DOM is a lightweight copy of actual DOM that uses diff algorithm for efficient updates
- How to install Node.js, npm, and Vue CLI
- How to create a new Vue 3 project using `vue create`
- Project structure of a Vue CLI application

Chapter 13 - Components

Vue 3 6 core topics with complete component code, theory, and live interactive demos

Simple Component · Data as Function · Local Registration · Global Registration · Dynamic Components · Component Events (mouseover/mouseout)

1. Simple Component - Reusable Building Block

Theory: Components are independent, reusable pieces of UI. In Vue 3, a component encapsulates HTML (template), logic (script), and scoped styles. Components help split the UI into manageable pieces and can be nested. Each component instance maintains its own state when data is a function.

- **Local registration:** import component and add to `components` option.
- **Template:** defines HTML structure with Vue directives.
- **Scoped styles:** CSS that applies only to this component.

```
// src/components/BasicComponent.vue
<template>
  <div class="my-component">
    <h1>This is coming from component</h1>
    <p>Hello! I am a reusable component.</p>
  </div>
</template>

<script>
export default {
  name: "BasicComponent",
};
</script>

<style scoped>
.my-component {
  border: 2px solid green;
  padding: 20px;
  margin: 10px;
  background-color: #e8f5e9;
  border-radius: 10px;
}
</style>

// App.vue
<template>
  <div id="app">
    <h1>My Vue 3 App</h1>
    <BasicComponent />
    <BasicComponent />
    <BasicComponent />
  </div>
</template>

<script>
import BasicComponent from "../components/BasicComponent.vue";
export default {
  name: "App",
  components: { BasicComponent },
};
```

```
</script>
```

Explanation: The `BasicComponent` defines a simple card with a heading and paragraph. `scoped` styles ensure the green border only affects this component. In `App.vue`, we import and register the component locally, then use it three times. Each instance is independent — demonstrating reusability.

🔗 Live Demo: Simple Component (three instances)

Each green box is a separate component instance — reusable and independent.

2. Data as a Function - Independent State Isolation

⚠️ **Critical Concept:** In Vue components, `data` must be a function that returns a fresh object. If it were a plain object, all instances would share the same reference, causing mutations to affect every component. This pattern guarantees each instance has its own private reactive scope.

`data() { return { count: 0 } } → correct.`

`data: { count: 0 } → wrong (Vue 3 throws warning).`

```
// src/components/ClickCounter.vue
<template>
  <div class="counter-box">
    <h3>Click Counter</h3>
    <p>You clicked me: <strong>{{ count }}</strong> times</p>
    <button @click="count++">Click Me!</button>
    <p class="small">Component ID: {{ componentId }}</p>
  </div>
</template>

<script>
export default {
  name: "ClickCounter",
  data() {
    return {
      count: 0,
      componentId: Math.random().toString(36).substr(2, 4),
    };
  },
};
</script>

<style scoped>
.counter-box {
  border: 2px solid orange;
  padding: 15px;
  margin: 10px;
  display: inline-block;
  width: 200px;
  text-align: center;
  background-color: #fff3e0;
  border-radius: 10px;
}
.counter-box button {
  background-color: orange;
  color: white;
  padding: 8px 15px;
```



```

border: none;
border-radius: 5px;
cursor: pointer;
}
.small { font-size: 10px; color: gray; margin-top: 10px; }
</style>

// App.vue
<template>
  <div id="app">
    <h1>Data in Components Demo</h1>
    <p>Each button works independently!</p>
    <ClickCounter />
    <ClickCounter />
    <ClickCounter />
  </div>
</template>

<script>
import ClickCounter from "../components/ClickCounter.vue";
export default {
  components: { ClickCounter }
};
</script>

```

Explanation: Each `ClickCounter` instance gets its own `{ count: 0, componentId }`. Clicking one counter increments only that instance's count — proving isolation. The unique `componentId` visually demonstrates that each component is a separate entity.

Live Demo: Three Independent Counters

Click any counter — others remain unchanged. Each has its own unique ID and count.

3. Local Registration - Scope & Encapsulation

Local Registration means a component is available only inside the component where it is registered (via `components` option). This prevents global conflicts, improves bundle splitting, and makes dependencies explicit. It's the recommended way for most components.

```

// src/components/SpecialButton.vue
<template>
  <button class="special-btn"> Special Button (Local Only)</button>
</template>

<script>
export default { name: "SpecialButton" };
</script>

<style scoped>
.special-btn {
  background-color: purple;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  margin: 5px;
  cursor: pointer;
}

```

```

    }
  </style>

  // src/components/ParentBox.vue
  <template>
    <div class="parent-box">
      <h3>Parent Component</h3>
      <p>This component can use SpecialButton because I registered it here:</p>
      <SpecialButton />
      <SpecialButton />
    </div>
  </template>

  <script>
  import SpecialButton from "../SpecialButton.vue";
  export default {
    name: "ParentBox",
    components: { SpecialButton }, // LOCAL registration
  };
  </script>

  <style scoped>
  .parent-box {
    border: 3px solid blue;
    padding: 20px;
    margin: 10px;
    border-radius: 10px;
  }
  </style>

  // App.vue
  <template>
    <div id="app">
      <h1>Local vs Global Registration</h1>
      <ParentBox />
      <!-- <SpecialButton /> would FAIL -->
    </div>
  </template>

  <script>
  import ParentBox from "../components/ParentBox.vue";
  export default {
    components: { ParentBox }
  };
  </script>

```

Explanation: SpecialButton is only registered inside ParentBox's components option. It works perfectly inside the parent, but trying to use <SpecialButton/> directly in App.vue would cause a "failed to resolve component" error. This encapsulation prevents naming collisions.

Live Demo: Parent with Locally Registered Child Button

The purple "Special Button" exists only inside the blue parent box — perfect demonstration of local scope.

4. Global Registration - One-Time, Use Everywhere

Global Components are registered using `app.component('name', Component)` in `main.js`. After registration, they can be used in any component template without importing. Best for high-frequency UI pieces (alerts, modals,

buttons). Vue 3 migrated from `Vue.component` to `app.component`.

```
// main.js
import { createApp } from "vue";
import App from "./App.vue";
import GlobalAlert from "./components/GlobalAlert.vue";

const app = createApp(App);
app.component("GlobalAlert", GlobalAlert); // GLOBAL REGISTRATION
app.mount("#app");

// src/components/GlobalAlert.vue
<template>
  <div class="alert-box" v-if="isVisible">
    <div class="alert-icon"></div>
    <div class="alert-content">
      <h3>{{ title }}</h3>
      <p>{{ message }}</p>
    </div>
    <button class="close-btn" @click="closeAlert">✖</button>
  </div>
</template>

<script>
export default {
  name: "GlobalAlert",
  props: { title: { type: String, default: "Notification" }, message: { type: String, default: "This is a global alert component!" } },
  data() { return { isVisible: true }; },
  methods: { closeAlert() { this.isVisible = false; } }
};
</script>

// App.vue – no import needed!
<template>
  <div id="app">
    <h1> Global Registration Demo</h1>
    <GlobalAlert title="Welcome!" message="Globally registered!" />
    <GlobalAlert title="Special Offer" message="Get 50% off!" />
  </div>
</template>
```

Explanation: After `app.component()`, the `GlobalAlert` component becomes globally available. You never need to import it in any component. The alert has internal state (`isVisible`) and a close button that hides it — demonstrating encapsulated behavior.

Live Demo: Globally Registered Alert Component

The alert is registered once globally. Click the ✖ button to close each alert independently.

5. Dynamic Components - `<component :is>`

Dynamic Components allow switching between components at runtime using the `<component :is="currentComponent">` element. Ideal for tabs, step wizards, or conditional layouts. The `is` attribute can be bound to a component object or registered name.

```

// src/components/HomePage.vue
<template>
  <div class="page home">
    <h2> Home Page</h2>
    <p>Welcome to the home page!</p>
  </div>
</template>
<style scoped>.page { padding: 20px; border-radius: 10px; } .home { background-color: #e3f2fd; }</style>

// src/components/AboutPage.vue
<template><div class="page about"><h2> About Page</h2><p>This is the about page content.</p></div></template>
<style scoped>.about { background-color: #f3e5f5; }</style>

// src/components/ContactPage.vue
<template><div class="page contact"><h2> Contact Page</h2><p>Contact us at: example@email.com</p></div></template>
<style scoped>.contact { background-color: #e8f5e9; }</style>

// src/components/DynamicTabs.vue
<template>
  <div>
    <div class="tabs">
      <button v-for="tab in tabs" :key="tab.name"
        @click="currentTab = tab.component"
        :class="{ active: currentTab === tab.component }">
        {{ tab.name }}
      </button>
    </div>
    <component :is="currentTab" />
  </div>
</template>

<script>
import HomePage from "./HomePage.vue";
import AboutPage from "./AboutPage.vue";
import ContactPage from "./ContactPage.vue";
export default {
  name: "DynamicTabs",
  data() {
    return {
      currentTab: HomePage,
      tabs: [
        { name: "Home", component: HomePage },
        { name: "About", component: AboutPage },
        { name: "Contact", component: ContactPage }
      ]
    };
  }
};
</script>

<style scoped>
.tabs { margin-bottom: 20px; }
.tabs button { padding: 10px 20px; margin-right: 10px; border: 1px solid #ddd; background: white; cursor: pointer; border-radius: 5px; }
.tabs button.active { background: #42b983; color: white; border-color: #42b983; }
</style>

// App.vue
<template>
  <div id="app">
    <h1>Dynamic Components Demo</h1>
    <p>Click the tabs to change the component</p>
    <DynamicTabs />
  </div>
</template>
<script>

```

```
import DynamicTabs from "../components/DynamicTabs.vue";
export default { components: { DynamicTabs } };
</script>
```

Explanation: The `currentTab` data property holds the currently active component. When a tab button is clicked, `currentTab` updates, and `<component :is="currentTab">` dynamically renders the selected component. Each page component has its own styled background, making tab switching visually clear.

Live Demo: Dynamic Tab Switching

Click the tabs — the component changes instantly using `<component :is>`. No routing needed!

6. Component Events - Mouse Interactions (@mouseover / @mouseout)

 **Component Events in Action:** Events allow components to react to user interactions. This demo uses `@mouseover` and `@mouseout` to dynamically change component state. Each card manages its own `cityName`, `isHovered`, and `textColor` — fully independent due to `data()` as a function.

Key Concepts: Native DOM events (`@mouseover`, `@mouseout`) are handled by component methods. Each instance maintains its own hover state, demonstrating perfect component isolation.

```
// src/components/InteractiveCard.vue
<template>
  <div class="interactive-card" @mouseover="handleMouseOver" @mouseout="handleMouseOut">
    <h2>
      Custom Component created by
      <span :style="{ color: textColor }">{{ cityName }}</span>
    </h2>
    <p v-if="isHovered" class="message">
      ☞ Mouse is over me! City changed to {{ cityName }}
    </p>
  </div>
</template>

<script>
export default {
  name: "InteractiveCard",
  data() {
    return {
      cityName: "Shanghai",
      isHovered: false,
      textColor: "blue",
    };
  },
  methods: {
    handleMouseOver() {
      this.cityName = "Guizhou";
      this.isHovered = true;
      this.textColor = "red";
    },
    handleMouseOut() {
      this.cityName = "Shanghai";
      this.isHovered = false;
      this.textColor = "blue";
    },
  },
}
```

```

    },
  };
</script>

<style scoped>
.interactive-card {
  border: 2px solid #42b983;
  padding: 20px;
  margin: 15px;
  text-align: center;
  background-color: #f0f9f4;
  border-radius: 10px;
  transition: all 0.3s;
  cursor: pointer;
  display: inline-block;
  width: 280px;
  vertical-align: top;
}
.interactive-card:hover {
  transform: scale(1.02);
  box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
}
.message {
  margin-top: 15px;
  padding: 10px;
  background-color: #fff3cd;
  border-radius: 5px;
  animation: fadeIn 0.3s;
}
@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}
</style>

// App.vue
<template>
  <div id="app">
    <h1>Component Events Demo</h1>
    <p><strong>Try this:</strong> Move your mouse over the boxes!</p>
    <InteractiveCard />
    <InteractiveCard />
    <InteractiveCard />
    <div class="info-box">
      <h3> What's happening?</h3>
      <ul>
        <li><strong>@mouseover</strong> - Changes city from Shanghai to Guizhou</li>
        <li><strong>@mouseout</strong> - Changes city back to Shanghai</li>
        <li>Each card works independently (data is a function)</li>
      </ul>
    </div>
  </div>
</template>

<script>
import InteractiveCard from "../components/InteractiveCard.vue";
export default {
  name: "App",
  components: { InteractiveCard }
};
</script>

<style scoped>
.info-box {
  margin-top: 30px;
  padding: 20px;
  background-color: #e8eaf6;

```

```
border-radius: 10px;
}
.info-box ul { text-align: left; }
</style>
```

Detailed Explanation:

- **@mouseover:** When mouse enters the card, `handleMouseOver()` changes `cityName` to "Guizhou", sets `isHovered = true`, and `textColor` to red. The message appears conditionally with `v-if="isHovered"`.
- **@mouseout:** Reverts all values to original state (Shanghai, blue, false).
- **Independent instances:** Because each card has its own `data()` return, hovering one card does NOT affect others — perfect demonstration of component isolation.
- **CSS animations:** Smooth scaling on hover and fade-in animation for the message enhance user experience.

🔗 Live Demo: Interactive Cards with Mouse Events

📌 **Try it:** Hover over any card → city changes from Shanghai → Guizhou, text turns red, and a message appears. Move mouse out → reverts to Shanghai. Each card behaves independently!

✂️ Vue 3 Components – Complete notes with ALL your provided component codes, theory, explanations & interactive demos.

Master component architecture: reusability, data isolation, registration strategies, dynamic rendering, and mouse-event-driven interactions.

Chapter 14 - VueJS Events

Vue 3 6 core topics with complete event handling code, theory, and live interactive demos

Events Introduction · Event Handling · Click Event · Event Modifiers (.once, .prevent) · Key Modifiers · Custom Events

1. Basic Click Event - Handling Button Clicks

Theory: Events in Vue.js are used to respond to user actions like clicks, mouse movements, keyboard presses, etc. The `v-on` directive (or its shorthand `@`) listens to DOM events and runs JavaScript when triggered.

- **Syntax:** `<button v-on:click="methodName">Click Me</button>`
- **Shorthand:** `<button @click="methodName">Click Me</button>`
- **Common events:** submit, keyup, drag, scroll, error, abort, mouseover, mouseout, load

```
// Vue 3 Component - ClickEventDemo.vue
<template>
  <div class="demo-container">
    <button @click="displayNumbers" class="btn-primary">
      Click Me to Add No.
    </button>
    <h2>Add Number 100 + 200 = {{ total }}</h2>
  </div>
</template>

<script>
export default {
  name: "ClickEventDemo",
  data() {
    return {
      num1: 50,
      num2: 100,
      total: ""
    };
  },
  methods: {
    displayNumbers(event) {
      console.log(event);
      this.total = this.num1 + this.num2;
    }
  }
};
</script>

<style scoped>
.btn-primary {
  background-color: #2196F3;
  color: white;
  padding: 8px 16px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 16px;
}
</style>
```


Explanation: When the button is clicked, the `displayNumbers` method is called. It adds `num1` (50) and `num2` (100) and stores the result in `total`. The `@click` shorthand is used instead of `v-on:click`. The method receives the native DOM event object as a parameter.

🔗 Live Demo: Basic Click Event

Click the button — it adds $50 + 100 = 150$. The console logs the click event object.

📄 2. .once Event Modifier - Execute Only Once

Theory: Event modifiers are directive postfixes denoted by a dot (e.g., `.once`, `.prevent`, `.stop`). The `.once` modifier allows an event to execute only once. After the first click, subsequent clicks have no effect.

Common modifiers: `.once` · `.prevent` · `.stop` · `.capture` · `.self` · `.passive`

```
// Vue 3 Component - OnceModifierDemo.vue
<template>
  <div class="demo-container">
    <button @click.once="buttonClickedOnce" :style="styleobj">
      Just once clickable
    </button>
    <p>Output: {{ clickNum }}</p>

    <button @click="buttonClicked" :style="styleobj">
      Click Me anytime
    </button>
    <p>Output: {{ clickNum1 }}</p>
  </div>
</template>
```

```
<script>
export default {
  name: "OnceModifierDemo",
  data() {
    return {
      clickNum: 0,
      clickNum1: 0,
      styleobj: {
        backgroundColor: "#2196F3",
        cursor: "pointer",
        padding: "8px 16px",
        verticalAlign: "middle",
        border: "none",
        borderRadius: "4px",
        color: "white"
      }
    };
  },
  methods: {
    buttonClickedOnce() {
      this.clickNum++;
    },
    buttonClicked() {
      this.clickNum1++;
    }
  }
};
```

```
</script>
```

Explanation: The first button uses `@click.once` — it increments `clickNum` only on the first click. Subsequent clicks do nothing. The second button uses regular `@click` — it increments `clickNum1` every time it's clicked.

🔗 Live Demo: `.once` Modifier vs Regular Click

Click "Just once clickable" multiple times — it only increments ONCE. The other button increments every click.

3. `.prevent` Event Modifier - Prevent Default Behavior

Theory: The `.prevent` modifier calls `event.preventDefault()` internally. It's used when you don't want the browser's default action to occur (e.g., following a link, submitting a form).

```
// Vue 3 Component - PreventModifierDemo.vue
<template>
  <div>
    <!-- WITH .prevent - won't navigate -->
    <a href="http://www.google.com" @click.prevent="clickMe" target="_blank" :style="styleobj">
      Click Me (With .prevent)
    </a>

    <!-- WITHOUT .prevent - will navigate -->
    <a href="http://www.google.com" @click="clickMeWithoutPrevent" target="_blank" :style="styleobj">
      Click Me (Without .prevent)
    </a>
  </div>
</template>

<script>
export default {
  name: "PreventModifierDemo",
  data() {
    return {
      styleobj: {
        color: "#4CAF50",
        marginLeft: "20px",
        fontSize: "30px",
        display: "inline-block",
        marginRight: "20px"
      }
    };
  },
  methods: {
    clickMe() {
      alert("Anchor tag is clicked! Navigation PREVENTED!");
    },
    clickMeWithoutPrevent() {
      alert("This will navigate to Google after this alert");
    }
  }
};
</script>
```

Explanation: The first link uses `@click.prevent` — it shows an alert but does NOT navigate to Google. The second link uses regular `@click` — it shows an alert AND then navigates to Google.

🔗 Live Demo: .prevent Modifier

Click the first link — alert appears, but no navigation. Click the second link — alert appears, THEN Google opens.

🖱️ 4. Key Modifiers - Handling Keyboard Events

Theory: Key modifiers are used to handle specific keyboard keys. Vue provides built-in key modifiers like `.enter`, `.tab`, `.delete`, `.esc`, `.space`, `.up`, `.down`, `.left`, `.right`. You can also chain multiple modifiers like `@keyup.ctrl.enter`.

```
// Vue 3 Component - KeyModifiersDemo.vue
<template>
  <div>
    <h3>Press ENTER after typing</h3>
    <input
      type="text"
      @keyup.enter="showInputValue"
      :style="styleobj"
      placeholder="Type your name here and press Enter"
      v-model="tempName"
    />
    <h3>Your name: {{ name }}</h3>

    <h3>Press Ctrl + Enter</h3>
    <input
      type="text"
      @keyup.ctrl.enter="showInputValueCtrl"
      :style="styleobj"
      placeholder="Type something and press Ctrl+Enter"
      v-model="tempMessage"
    />
    <h3>Message: {{ message }}</h3>

    <div class="key-list">
      <span class="key-badge"> .enter</span>
      <span class="key-badge"> .tab</span>
      <span class="key-badge"> .delete</span>
      <span class="key-badge"> .esc</span>
      <span class="key-badge"> .space</span>
      <span class="key-badge"> .up</span>
      <span class="key-badge"> .down</span>
      <span class="key-badge"> .left</span>
      <span class="key-badge"> .right</span>
    </div>
  </div>
</template>

<script>
export default {
  name: "KeyModifiersDemo",
  data() {
    return {
      name: "",
      tempName: "",
      message: "",
      tempMessage: "",
    }
  }
}
```

```

styleobj: {
  width: "100%",
  padding: "12px 20px",
  margin: "8px 0",
  boxSizing: "border-box",
  border: "2px solid #ccc",
  borderRadius: "4px",
  fontSize: "16px"
}
};
},
methods: {
  showInputValue(event) {
    this.name = event.target.value;
    console.log("Enter pressed:", this.name);
  },
  showInputValueCtrl(event) {
    this.message = event.target.value;
    console.log("Ctrl+Enter pressed:", this.message);
  }
}
};
</script>

```

Explanation: The first input uses `@keyup.enter` — the value appears ONLY when you press the Enter key. The second input uses `@keyup.ctrl.enter` — the value appears only when you press Ctrl + Enter together.

⚡ Live Demo: Key Modifiers

Type in the first box — nothing happens until you press ENTER. Type in the second box — nothing happens until you press CTRL+ENTER.

5. Custom Events - Child to Parent Communication

Theory: Custom events allow child components to communicate with parent components. The child uses `$emit('event-name', data)` to send data upward. The parent listens using `@event-name="handlerMethod"`. This is the standard way for child-to-parent communication in Vue.

```

// Vue 3 Child Component - LanguageButton.vue
<template>
  <button @click="displayLanguage" class="language-btn">
    <span style="font-size:25px;">{{ item }}</span>
  </button>
</template>

<script>
export default {
  name: "LanguageButton",
  props: ["item"],
  methods: {
    displayLanguage() {
      this.$emit('showLanguage', this.item);
    }
  }
};
</script>

```

```
// Parent Component - CustomEventsDemo.vue
<template>
  <div>
    <p style="font-size:25px;">
      Language displayed : <b>{{ languageClicked }}</b>
    </p>
    <div class="languages-grid">
      <LanguageButton
        v-for="(item, index) in languages"
        :key="index"
        :item="item"
        @showLanguage="languageDisp"
      />
    </div>
  </div>
</template>

<script>
import LanguageButton from "./LanguageButton.vue";

export default {
  name: "CustomEventsDemo",
  components: { LanguageButton },
  data() {
    return {
      languageClicked: "",
      languages: ["Java", "C", "C++", "Python", "Javascript", "Angular", "Data Structure", "jQuery"]
    };
  },
  methods: {
    languageDisp(language) {
      this.languageClicked = language;
      console.log("Language selected:", language);
    }
  }
};
</script>
```

Explanation: The child component (LanguageButton) receives a item prop and emits a custom showLanguage event when clicked. The parent listens with @showLanguage="languageDisp" and updates languageClicked to display the selected language.

✂ Live Demo: Custom Events (Parent-Child Communication)

Click any language button — the selected language appears above. This is child-to-parent communication using \$emit.

6. Complete Event Handling - Mouse Events

Theory: Vue can handle any DOM event including mouseover, mouseout, mousemove, etc. This demo shows mouse enter/leave events that change component state dynamically.

```
// Vue 3 Component - InteractiveCard.vue
<template>
  <div class="interactive-card" @mouseover="handleMouseOver" @mouseout="handleMouseOut">
    <h2>
```

```

        Custom Component created by
        <span :style="{ color: textColor }">{{ cityName }}</span>
    </h2>
    <p v-if="isHovered" class="message">
        ☞ Mouse is over me! City changed to {{ cityName }}
    </p>
</div>
</template>

<script>
export default {
  name: "InteractiveCard",
  data() {
    return {
      cityName: "Shanghai",
      isHovered: false,
      textColor: "blue"
    };
  },
  methods: {
    handleMouseOver() {
      this.cityName = "Guizhou";
      this.isHovered = true;
      this.textColor = "red";
    },
    handleMouseOut() {
      this.cityName = "Shanghai";
      this.isHovered = false;
      this.textColor = "blue";
    }
  }
};
</script>

<style scoped>
.interactive-card {
  border: 2px solid #42b983;
  padding: 20px;
  margin: 15px;
  text-align: center;
  background-color: #f0f9f4;
  border-radius: 10px;
  cursor: pointer;
  display: inline-block;
  width: 280px;
}
.message {
  margin-top: 15px;
  padding: 10px;
  background-color: #fff3cd;
  border-radius: 5px;
}
</style>

// App.vue
<template>
  <div>
    <h1>Component Events Demo</h1>
    <p><strong>Try this:</strong> Move your mouse over the boxes!</p>
    <InteractiveCard />
    <InteractiveCard />
    <InteractiveCard />
  </div>
</template>

<script>
import InteractiveCard from "../components/InteractiveCard.vue";

```

```
export default {  
  components: { InteractiveCard }  
};  
</script>
```

Explanation: When mouse enters the card (@mouseover), city changes to "Guizhou", text turns red, and a message appears. When mouse leaves (@mouseout), city reverts to "Shanghai", text turns blue, and message disappears. Each card operates independently.

✂ Live Demo: Mouse Events (mouseover/mouseout)

☐ Hover over any card → city changes from Shanghai → Guizhou, text turns red. Move mouse out → reverts to Shanghai.

✂ Vue 3 Events – Complete notes with all event handling codes, theory, explanations & interactive demos.
Master event handling: click events, event modifiers (.once, .prevent), key modifiers (enter, ctrl+enter), custom events (\$emit), and mouse events (mouseover/mouseout).

Chapter 15 - Rendering, Transition & Animation

Vue 3 8 core topics with complete rendering, transition and animation code, theory, and live interactive demos

v-if · v-else · v-show · v-for · Fade Transition · ShiftX Transition · Rotate Animation · Custom Transition Classes (Animate.css)

1. v-if Directive - Conditional Rendering

Theory: The `v-if` directive is used to conditionally render a block. The element with `v-if` will only be produced if the directive's expression returns a `true` value. If the expression produces an invalid result (`null`, `0`, empty string, `false`), the element is deleted from the DOM.

- **v-if** - Renders element only if condition is true
- **v-else** - Renders element when `v-if` is false (must immediately follow `v-if`)
- **v-else-if** - Chains multiple conditions

```
// Vue 3 Component - VIfDemo.vue
<template>
  <div id="databinding">
    <button @click="showdata" :style="styleobj">Click Me</button>
    <span style="font-size:25px;"><b>{{ show }}</b></span>
    <h1 v-if="show">This is h1 tag</h1>
    <h2>This is h2 tag (always visible)</h2>
  </div>
</template>

<script>
export default {
  name: "VIfDemo",
  data() {
    return {
      show: true,
      styleobj: {
        backgroundColor: "#2196F3",
        cursor: "pointer",
        padding: "8px 16px",
        verticalAlign: "middle",
        border: "none",
        borderRadius: "4px",
        color: "white"
      }
    };
  },
  methods: {
    showdata() {
      this.show = !this.show;
    }
  }
};
</script>
```

Explanation: When `show` is `true`, the `h1` tag is displayed. Clicking the button toggles `show` to `false`, and Vue removes the `h1` tag from the DOM entirely. The `h2` tag is always visible because it has no directive.

⚡ Live Demo: v-if Directive

Click the button — the h1 tag disappears completely from the DOM when show becomes false.

2. v-if and v-else - Toggle Between Elements

Theory: The `v-else` directive must immediately follow a `v-if` or `v-else-if` element. It renders its content only when the preceding condition is false.

```
// Vue 3 Component - VIfElseDemo.vue
<template>
  <div id="databinding">
    <button @click="showdata" :style="styleobj">Click Me</button>
    <span style="font-size:25px;"><b>{{ show }}</b></span>
    <h1 v-if="show">This is h1 tag</h1>
    <h2 v-else>This is h2 tag</h2>
  </div>
</template>

<script>
export default {
  name: "VIfElseDemo",
  data() {
    return {
      show: true,
      styleobj: {
        backgroundColor: "#2196F3",
        cursor: "pointer",
        padding: "8px 16px",
        verticalAlign: "middle",
        border: "none",
        borderRadius: "4px",
        color: "white"
      }
    };
  },
  methods: {
    showdata() {
      this.show = !this.show;
    }
  }
};
</script>
```

Explanation: When `show` is true, the `h1` tag is displayed. When `show` becomes false, the `h1` tag is removed and the `h2` tag (with `v-else`) is displayed instead.

⚡ Live Demo: v-if and v-else

Click the button — toggles between "This is h1 tag" and "This is h2 tag".

3. v-show Directive - CSS Display Toggle

Theory: `v-show` behaves similarly to `v-if` but with a key difference: `v-if` removes/adds the element from the DOM, while `v-show` only toggles the element's `display` CSS property (`display: none` when false). The element always remains in the DOM.

```
// Vue 3 Component - VShowDemo.vue
<template>
  <div id="databinding">
    <button @click="showdata" :style="styleobj">Click Me</button>
    <span style="font-size:25px;"><b>{{ show }}</b></span>
    <h1 v-if="show">This is h1 tag (v-if)</h1>
    <h2 v-else">This is h2 tag (v-else)</h2>
    <div v-show="show">
      <b>V-Show:</b>
      
    </div>
  </div>
</template>

<script>
export default {
  name: "VShowDemo",
  data() {
    return {
      show: true,
      styleobj: {
        backgroundColor: "#2196F3",
        cursor: "pointer",
        padding: "8px 16px",
        verticalAlign: "middle",
        border: "none",
        borderRadius: "4px",
        color: "white"
      }
    };
  },
  methods: {
    showdata() {
      this.show = !this.show;
    }
  }
};
</script>
```

Explanation: The image inside `v-show` is always in the DOM. When `show` is false, it's hidden with `display: none`. When true, it's visible. Inspect the element to see it's still present in the DOM!

🔗 Live Demo: v-show Directive

Click the button — the image hides/shows using CSS display property. It is NEVER removed from the DOM (unlike `v-if`).

4. v-for Directive - List Rendering

Theory: The `v-for` directive renders a list of items based on an array. The syntax is `item in items` where `items` is the source array and `item` is the alias for each element. **Note:** Vue 3 requires a `:key` attribute for each rendered item.

```
// Vue 3 Component - VForDemo.vue
<template>
  <div id="databinding">
    <input
      type="text"
      @keyup.enter="showinputvalue"
      :style="styleobj"
      placeholder="Enter Fruits Names"
    />
    <h1 v-if="items.length > 0">Display Fruits Name</h1>
    <ul>
      <li v-for="(a, index) in items" :key="index">{{ a }}</li>
    </ul>
  </div>
</template>

<script>
export default {
  name: "VForDemo",
  data() {
    return {
      items: [],
      styleobj: {
        width: "30%",
        padding: "12px 20px",
        margin: "8px 0",
        boxSizing: "border-box",
        border: "2px solid #ccc",
        borderRadius: "4px",
        fontSize: "16px"
      }
    };
  },
  methods: {
    showinputvalue(event) {
      if (event.target.value.trim()) {
        this.items.push(event.target.value.trim());
        event.target.value = "";
      }
    }
  }
};
</script>
```

Explanation: The input field captures fruit names. When Enter is pressed, the value is added to the `items` array. `v-for` iterates over the array and displays each item as a list item. The `:key="index"` helps Vue track each item uniquely.

✂ Live Demo: v-for Directive

Type a fruit name and press Enter — it gets added to the list. Each new item appears automatically!

5. Fade Transition - Smooth Opacity Change

Theory: Vue provides a `<transition>` wrapper component to apply animations when elements are inserted, updated, or removed. The `name` attribute defines the prefix for transition classes. For fade transition, Vue applies `.fade-enter-active`, `.fade-leave-active`, etc.

```
// Vue 3 Component - FadeTransitionDemo.vue
<template>
  <div id="databinding">
    <p>Click at the below button to see transition effect.</p>
    <button @click="show = !show" class="btn-primary">Click Here</button>

    <transition name="fade">
      <p v-show="show" :style="styleobj">This is a Fade Transition Example</p>
    </transition>
  </div>
</template>

<script>
export default {
  name: "FadeTransitionDemo",
  data() {
    return {
      show: true,
      styleobj: {
        fontSize: "30px",
        color: "red"
      }
    };
  }
};
</script>

<style scoped>
.fade-enter-active,
.fade-leave-active {
  transition: opacity 3s;
}

.fade-enter,
.fade-leave-to {
  opacity: 0;
}
</style>
```

Explanation: The `<transition name="fade">` wraps the element. CSS classes define a 3-second opacity transition. When element enters, it fades in from opacity 0 to 1. When leaving, it fades out from 1 to 0.

🔗 Live Demo: Fade Transition (3 seconds)

Click the button — the text fades out over 3 seconds. Click again — it fades back in over 3 seconds.

➔ 6. ShiftX Transition - Slide on X-axis

Theory: Transitions can animate CSS properties like `transform`. This demo uses `translateX(100px)` to shift an image 100 pixels to the right when entering or leaving.

```
// Vue 3 Component - ShiftXTransitionDemo.vue
<template>
  <div id="databinding">
    <p>Click at the below button to see transition effect.</p>
    <button @click="show = !show" class="btn-primary">Click Here</button>

    <transition name="shiftx">
      <div v-show="show">
        
      </div>
    </transition>
  </div>
</template>

<script>
export default {
  name: "ShiftXTransitionDemo",
  data() {
    return {
      show: true
    };
  }
};
</script>

<style scoped>
.shiftx-enter-active,
.shiftx-leave-active {
  transition: all 2s ease-in-out;
}

.shiftx-enter,
.shiftx-leave-to {
  transform: translateX(100px);
  opacity: 0;
}
</style>
```

Explanation: When the element enters, it starts 100px to the right (opacity 0) and slides into place. When leaving, it slides 100px to the right and fades out simultaneously.

✂ Live Demo: ShiftX Transition (Slide)

Click the button — the image slides 100px to the right while fading. Click again — it slides back from the right.

7. Rotate Animation - Keyframes Animation

Theory: Vue transitions can use CSS @keyframes for complex animations. This demo uses a 2-second animation that rotates an image 360 degrees on the X-axis (3D flip).

```
// Vue 3 Component - RotateAnimationDemo.vue
<template>
  <div id="databinding">
```

```

<p>Click at the below button to see the animation effect.</p>
<button @click="show = !show" class="btn-primary">Click Here</button>

<transition name="shiftx">
  <div v-show="show">
    
  </div>
</transition>
</div>
</template>

<script>
export default {
  name: "RotateAnimationDemo",
  data() {
    return {
      show: true
    };
  }
};
</script>

<style scoped>
.shiftx-enter-active {
  animation: shift-in 2s;
}

.shiftx-leave-active {
  animation: shift-in 2s reverse;
}

@keyframes shift-in {
  0% { transform: rotateX(0deg); opacity: 0; }
  25% { transform: rotateX(90deg); opacity: 0.25; }
  50% { transform: rotateX(120deg); opacity: 0.5; }
  75% { transform: rotateX(180deg); opacity: 0.75; }
  100% { transform: rotateX(360deg); opacity: 1; }
}
</style>

```

Explanation: @keyframes shift-in defines a multi-step animation. When entering, it runs forward (0° to 360°). When leaving, it runs in reverse (360° to 0°), creating a 3D flip effect.

🔗 Live Demo: Rotate Animation (3D Flip)

Click the button — the image performs a 3D rotation (0° → 360°) over 2 seconds. Click again — it rotates back from 360° → 0°.

8. Custom Transition Classes - Animate.css Integration

Theory: Vue allows custom transition classes like enter-active-class and leave-active-class to integrate with external CSS animation libraries (e.g., Animate.css). This eliminates the need to write custom CSS animations.

Custom class attributes: enter-class · enter-active-class · enter-to-class · leave-class · leave-active-class ·

```
// Vue 3 Component - CustomTransitionDemo.vue
<template>
  <div id="animation" style="text-align:center">
    <button @click="show = !show" class="btn-primary">
      <span style="font-size:25px;">Click Here</span>
    </button>

    <transition
      enter-active-class="animate__animated animate__swing"
      leave-active-class="animate__animated animate__bounceOut"
    >
      <p v-if="show" class="animation-text">
        <span style="font-size:25px;">See the animation effect</span>
      </p>
    </transition>
  </div>
</template>

<script>
export default {
  name: "CustomTransitionDemo",
  data() {
    return {
      show: true
    };
  }
};
</script>

<style scoped>
.animation-text {
  display: inline-block;
  margin-top: 20px;
  padding: 20px 40px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  border-radius: 15px;
  color: white;
  box-shadow: 0 10px 30px rgba(0,0,0,0.2);
}
</style>
```

Explanation: Instead of writing custom CSS animations, we use Animate.css classes. `enter-active-class="animate__animated animate__swing"` applies a swing animation when entering. `leave-active-class="animate__animated animate__bounceOut"` applies a bounce-out animation when leaving.

🔗 Live Demo: Custom Transition Classes (Animate.css)

Click the button — text swings like a pendulum when appearing, bounces out when disappearing. No custom CSS animations needed!

Transition Classes Reference

Standard Classes (prefixed with transition name):

- `v-enter` – Starting state before element is added

- `v-enter-active` – Active state during entering (defines delay, duration, easing)
- `v-enter-to` – Ending state for entering (Vue 2.1.8+)
- `v-leave` – Starting state for leaving transition
- `v-leave-active` – Active state during leaving (defines delay, duration, easing)
- `v-leave-to` – Ending state for leaving (Vue 2.1.8+)

Custom Class Attributes:

- `enter-class` – Custom class for entering start state
- `enter-active-class` – Custom class for entering active state
- `enter-to-class` – Custom class for entering end state
- `leave-class` – Custom class for leaving start state
- `leave-active-class` – Custom class for leaving active state
- `leave-to-class` – Custom class for leaving end state

✂ **Vue 3 Rendering, Transition & Animation** – Complete notes with all conditional rendering, list rendering, transitions, and animation codes.

Master rendering: `v-if` vs `v-show` differences, `v-for` with `:key`, fade/slide/rotate transitions, and custom animation classes with `Animate.css`.